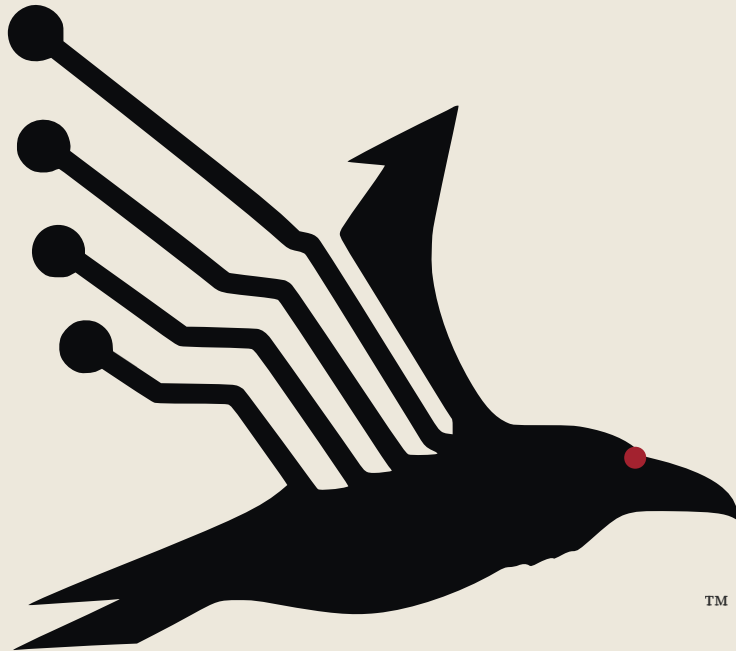




Ravenhelm Linear Operating Method

*A tiered, human-centric operating model for running a software
team on Linear.*

R L O M • v 1 . 0 . 2

Ravenhelm LLC

Ravenhelm Linear Operating Method – v1.0.2 (2026-10-06) · build 6e4ad88 ·
2026-10-06

Copyright © 2026 Ravenhelm LLC. All rights reserved except as expressly granted in the RLOM
Community License 1.0 ([full text at the back of this edition](#)).

Free to read and to implement internally. Verbatim, unmodified copies may be redistributed
for non-commercial, personal, or educational purposes. Modified versions, commercial repub-
lication, third-party RLOM services, training, certification, and commercial incorporation re-
quire prior written permission from Ravenhelm LLC.

Inquiries: nate@ravenhelm.co · ravenhelm.co · Source: github.com/nwalker85/rlom-method

Set in Source Serif 4 and IBM Plex Mono.

Contents

How to read it 5

The principles at a glance 6

01 What This Is and Who It's For 7

Who this is for · What the Method is · The shape, briefly · The promise · What this is not

02 The Seven Operating Layers 12

Why seven layers, not two · Initiatives — the objective layer · Projects — the main execution container · Issues — deliverable-sized work units · Views — the operator's daily control panel · Intake — the entry point for new work · Delivery evidence — the substrate for Validation · Analytics — the reporting substrate · Layer-to-capability map · Closing

03 The Numbered Principles 18

How to use the principles · Group A — The work graph (the shape) · Group B — Evidence and validation · Group C — Workflow canon · Group D — The Operator · Group E — The operating surface · Group F — Cadence · Group G — Intake and feedback · Group H — System hygiene · Group I — Reporting · Group J — Releases · Closing

04 The PM Operator Role 35

What the Operator owns · The PM Operator practice ladder · The daily and weekly shape · Mapping upward and downward

05 The Operating Surface: Labels, Views, Templates 40

Labels are an index, not an organizational scheme · Views are the daily control panel · Templates lower the cost of doing the right thing · Relations and recurring issues · The overbuild test

06 Tier 1: Linear Free Install 47

What Free actually gives you · Week one install · Growing into the rest — still on Free · What you have at the end of week one · Ceiling signal

07 Tier 2: Linear Basic Upgrade 55

What Basic actually adds · The misconception to drop · The upgrade is administrative, not architectural · Admin roles — the one genuine new lever · Standing up the second team · Ceiling signal — how to know Basic is no longer enough

08 Tier 3: Linear Business and the Edge of Self-Install 60

The shift Business represents · Insights — the first analytics layer · Triage Intelligence · Linear Asks · SLAs · Agent surfaces — agents as workspace participants · The escalation — agents that write and ship code · Team boundaries, guests, and access controls · Ceiling signal — when the Operator's bandwidth runs out

09

Tier 4: Linear Enterprise and the Method as Operating System

70

The shift Enterprise represents · The federated operating model — hub, spokes, and a community of practice · Identity and the audit trail — SSO, SCIM, restrictions, audit log · Sub-teams, private teams, and guests — the federation’s structure · Dashboards and data out — the cross-team instrument panel · Enterprise intake — Asks web forms, Salesforce, Gong · Agent guidance and Skills across the federation · Ceiling signal — when the method becomes the operating system

10

Workflow Canon and the Validation Gate

The nine states · Why Validation is load-bearing · The five evidence types · The three-person team flow · Nine states is the contract

11

Cycles, Roadmaps & Cadence

Cadence is the spine; cycles are an overlay · When to turn cycles on · Running a cycle: capacity and the seventy-percent rule · Roadmaps: the timeline over initiatives and projects · Plan-of-Record vs. What-If · Rhythm and horizon, not new bureaucracy

12

Code Host Integration: GitHub Primary, GitLab Parallel

Branch and PR conventions · PR opened triggers In Review · PR merged triggers Validation — not Done · Release events trigger Done · Naming releases: Semantic Versioning · The 3-person team installs both · Self-hosted forges — the webhook bridge · Install order · What not to do

13

Reporting Up: Project Updates, Initiative Health, and the Monthly Review

The three cadences and the nesting · The weekly project update · The biweekly initiative update · The monthly operating review · Computed health (the RAG operating model) · Pulse (the consumption layer) · Friday afternoon, in practice · What reporting is for

14

The Assessment: What This Document Doesn't Cover

What this document gave you · What this document deliberately did not give you · The Ravenhelm Operating Assessment · Engaging

A

The Operations-Team Variant

Supply-side and demand-side — and who should skip this appendix · Intake as a first-class layer · The Service Catalog · Governance cadence · Service SLAs · CoE-grade field discipline · Two optional label axes · Computed RAG governance

How to read it

IN ORDER

The chapters build on each other. Chapter 3 (Principles) is cited by every later chapter as **P-#**. In this edition every citation is a link to the principle, and every principle links back to the chapters that cite it.

BY ROLE – THE INSTALL PATH

A PM at a 3-person team starting on Linear Free should read these and skim the rest.

- 01 What This Is
- 02 The Seven Operating Layers
- 04 The PM Operator Role
- 05 The Operating Surface
- 06 Tier 1 · Linear Free
- 10 Workflow Canon
- 13 Reporting Up

BY NEED – DEEPENING PRACTICE

A team already on Basic wanting to deepen practice reads these first.

- 10 Workflow Canon
- 11 Cycles, Roadmaps & Cadence
- 12 Code Host Integration
- 13 Reporting Up

NAVIGATING THIS PDF

The bookmark panel carries every chapter, section, and principle. Chapter and appendix references in running text are links. Chapter openers list the principles each chapter cites.

The principles at a glance

GROUP A — THE WORK GRAPH (THE SHAPE)	
P-01	Linear is the active work graph, not an archive 19
P-02	Initiatives are the objective layer 20
P-03	Projects are the main execution container 20
P-04	Issues are deliverable-sized 21
P-05	Documentation owns deep context 21
GROUP B — EVIDENCE AND VALIDATION	
P-06	Validation separates implemented from trusted 22
P-07	Evidence beats assertion 22
P-08	The code host proves implementation 23
P-09	Production work does not go Done before release evidence 23
GROUP C — WORKFLOW CANON	
P-10	The workflow canon is nine states 24
P-11	Stage labels carry the things that look like states but aren't 24
P-12	Prefer parent issues with sub-issues over giant single issues 25
GROUP D — THE OPERATOR	
P-13	The Operator owns priority, taste, risk, and real-world commitments 25
P-14	Agents are executors, not owners 26
P-15	Agent delegation requires acceptance criteria and a validation command 26
GROUP E — THE OPERATING SURFACE	
P-16	Labels exist to serve views 27
P-17	Templates lower the cost of doing the right thing 27
P-18	Locality wins 28
GROUP F — CADENCE	
P-19	Active projects get a status update every week 28
P-20	Triage has a seven-day SLA 29
P-21	Initiative updates run biweekly; the monthly review rolls initiatives up to Leadership 29
GROUP G — INTAKE AND FEEDBACK	
P-22	Customer feedback does not directly become scope 30
P-23	The system of record is Linear 30
GROUP H — SYSTEM HYGIENE	
P-24	The overbuild test 31
P-25	Verify what you enforce 31
P-26	Capacity commits to seventy percent 32
GROUP I — REPORTING	
P-27	Leadership consumes summaries, not raw Linear 32
P-28	The evidence chain holds 33
GROUP J — RELEASES	
P-29	Releases are named with Semantic Versioning 33

What This Is and Who It's For

INSTALL PATH

PRINCIPLES CITED

P-1

P-2

P-6

P-13

P-16

P-27

P-28

You opened Linear this morning and the Backlog view returned 217 issues. Twelve of them were filed by you, in your own voice, six months ago. The rest came from Engineer A and Engineer B over the last two quarters, plus a slow drip from a Slack channel that you bolted to Linear in a hopeful afternoon and have not looked at since. There is no project rollup that means anything. There are four projects, two of which are abandoned, one of which is named “misc,” and one of which is the only honest container in the workspace. Yesterday someone on the leadership team asked you, in a meeting, “what’s the status of the platform work?” — and you produced an answer that you knew, while saying it, was a guess. You went back to your desk and tried to build a view that would let you give a better answer next time. You spent ninety minutes on it. You closed the tab.

This is the situation the Method is built for. It is not a transformation. It is not a re-org. It is a method you can install in a week and grow into over a quarter.

Who this is for

The reader is a single PM running a 3-person team — call them the Operator, Engineer A, and Engineer B — who reports up to a “leadership team” that wants status they can trust without being told to log into the tool. The Operator is technical enough to install Linear, wire up the GitHub or GitLab integration, write a few label conventions, and follow a workflow diagram. They are not senior enough to have ever designed an operating model from scratch, and they shouldn’t have to be. That work has been done. What follows is the result.

This Method is also useful to a slightly larger reader — a PM on a 5- or 6-person team, or a tech lead doing PM work on the side — but it is calibrated to the 3-per-

son case. Everything scales up. Some things scale down to two people and a contractor; below that the overhead exceeds the value and the reader should keep using a single Backlog view and a wiki page until they don't fit.

One reader shape is different enough to get its own treatment: a team whose board is other people's requests — an operations, PMO, or internal Center-of-Excellence team. That team is *demand-side* (work is requested through it) where the core Method assumes *supply-side* (the team builds its own board). **Appendix A** adapts the Method for that case — the intake, service-catalog, and governance front-half — without restating the shared execution core.

The Method assumes you have access to Linear and the authority to configure it. It does not assume Basic or Business. Ten of the thirteen chapters that follow are usable on Linear Free immediately. The three that aren't — Chapters 7, 8, and 9 — describe the tier ceilings as honest signals, not artificial paywalls, and explain what to do when you hit them.

What the Method is

The Ravenhelm Linear Operating Method is a tiered, human-centric operating model that lives inside Linear's actual primitives. Three properties define it.

Tiered. The Method comes in four tiers — Free, Basic, Business, Enterprise — that map to Linear's own pricing tiers because the capability ceilings are real. A 3-person team can install most of the Method on Free in a week: initiatives, Customer Requests, cycles, Releases, Pulse, and the full nine-state workflow all live on Free. The Basic tier adds no new capability — it lifts Free's ceilings (more teams, unlimited issues, admin roles); teams reach for it when they outgrow the 2-team or 250-issue limit, not for a feature. The Business tier is the first real capability jump: Insights, Triage Intelligence, Asks, SLAs, and agent surfaces — the edge of what a single Operator can install without help. The Enterprise tier is not more capability for one team; it is running the Method across many teams at once — SSO and SCIM, sub-teams, and workspace Dashboards. Each tier upgrade should be triggered by a specific operating pain, not by a sales calendar.

Human-centric. The Operator owns priority, taste, risk, and real-world commitments (**P-13**). No automation, no agent layer, no scheduled job decides what matters next. Tools surface candidates; the Operator decides. This is the load-bearing principle for the human role and the reason the Method does not collapse into a workflow engine. Framework-heavy methodologies — SAFe, Scrum-at-scale, the multi-month "agile transformation" engagement — reorganize the

team around the framework and then negotiate with reality. The Method reorganizes the workspace around the Operator’s judgment and then negotiates with Linear. The judgment stays where it belongs.

Grounded in Linear’s actual primitives. Initiatives, projects, issues, views, Triage, project updates, the GitHub or GitLab integration, Releases. The Method does not invent a sidecar database, a parallel state machine, or a custom field schema that fights the tool. Initiatives are the objective layer (P-2). Projects are the execution container. Issues are deliverable-sized. Linear is the active work graph (P-1), not an archive — content that doesn’t represent active or imminent execution belongs in a wiki or a docs/ directory, not a stuck Backlog issue from Q2.

The shape, briefly

Chapter 2 covers the seven operating layers in full: initiatives, projects, issues, views, intake, delivery evidence, and analytics. They are listed here only so the reader knows what the rest of the document is structuring. The layers are not a stack in the architectural sense — they are seven surfaces that the Operator touches in different rhythms. Issues are touched daily. Projects weekly. Initiatives biweekly. Analytics monthly. Delivery evidence (the GitHub or GitLab side) is touched continuously by Engineer A and Engineer B, and the Method’s job is to make sure that evidence flows back into Linear without the Operator copying and pasting anything.

Chapter 4 covers the PM Operator practice ladder — five levels of operating maturity that describe what the Operator should be able to do at each stage, from “I can keep the Backlog clean and the workflow honest” through “I can produce a monthly operating review that Leadership uses to make decisions.” Most readers of this document are at level 1 or 2 today; the goal of the Method is level 3 within a quarter and level 4 within two. Level 5 is rare and is generally where an agent layer starts to earn its keep.

The promise

You can install the Free tier in one week. You can be producing project updates that Leadership actually reads (P-27) in two. You can have a workflow that distinguishes implemented code from trusted code (P-6) within three — Chapter 10 covers this in depth, and Validation is the principle most teams skip and most regret skipping. Within a quarter, the workspace will be producing more signal

than the Operator has to manually generate, and the question “what’s the status of the platform work?” will have a one-paragraph answer that the Operator can recite from memory because the answer is the same as the most recent project update.

The Method is not a guarantee. It is a contract. The Operator brings judgment and one week of install work. The Method brings the structure that makes that judgment legible to a team and a leadership audience that don’t live inside the workspace.

AUGMENTATION SURFACE • Workspace Onboarding

Where an agent layer makes this work better: a brand-new workspace install benefits from automatic surfacing of the things the Operator would otherwise discover one tab at a time — count of stuck Backlog issues, age of the stalest item, projects with no owner or no update in 30 days, issues with no assignee, labels referenced by no view (P-16). The agent produces the inventory; the Operator opens the document and reads it before triaging.

What the PM keeps: deciding which surfaced gaps deserve the first week of attention, and which are acceptable noise to leave alone until the workspace is otherwise honest.

What this is not

This is not an agent framework. The Method names where an agent layer helps — the Augmentation Surface boxes in each chapter are precise about that — but it does not ship prompts, orchestration patterns, or build instructions for an AI PM. Those are Ravenhelm product.

This is not a prompt library. It is an operating model. The two are different artifacts and confusing them produces a workspace organized around the prompts rather than the work.

This is not a Linear sales document. The Method is calibrated to Linear because Linear’s primitives match the shape of the work. A team that already has a different tool installed and operating well does not need to migrate. A team that is choosing between tools should evaluate them on whether the seven operating layers can be installed without fighting the product, not on feature checklists.

This is not a transformation. There is no kickoff workshop, no maturity assessment that produces a 60-page deck, no quarterly steering committee. The Oper-

ator reads, installs, operates, and reports. The Method either earns its keep in the first month or it doesn't.

WHERE THIS GETS HARD

The Method assumes a single Operator feeding a single engineering team and reporting to a single leadership audience. When the Operator gains a second engineering team, or when Leadership starts asking for metrics the workspace isn't structured to produce yet — cross-team throughput, initiative-level forecast confidence, evidence chains that span three repos and two release pipelines (P-28) — the install becomes an integration problem, not a configuration problem. The shape stays right; the wiring gets specific.

Ravenhelm assesses your operating surface and installs the Method tuned to your team, your code host, and your reporting structure. nate@ravenhelm.co

The Seven Operating Layers

INSTALL PATH	PRINCIPLES CITED	P-1	P-2	P-3	P-4	P-5	P-6	P-7	P-8	P-12
P-16	P-18	P-19	P-20	P-21	P-22	P-23	P-28			

You installed Linear yesterday. You created a team, accepted Engineer A and Engineer B, and now you’re looking at an empty workspace with at least ten primitives staring back at you: initiatives, projects, issues, cycles, milestones, views, Triage, Asks, Customer Requests, releases, labels, documents. The product is opinionated, but it doesn’t tell you which primitives carry which weight. So you do what most operators do: you create a few projects, dump issues into them, and within a week you can’t tell which view is the one you check on Mondays.

The Method’s first job is to give you a structural map. Linear has many features; the Method uses seven layers. Everything else is supporting machinery that belongs to one of the seven.

Why seven layers, not two

Jira gives you epics and tickets. Trello gives you cards on a board. Both leave the objective layer and the evidence layer to your imagination — you bolt OKRs onto a spreadsheet and your “done” definition onto a Slack channel. Linear ships both natively: initiatives sit above projects as a real objective container (P-2), and the integration surface with the code host ships evidence into the issue automatically (P-8). The Method’s seven layers are not a methodology imposed on Linear; they are the layers Linear already exposes, named honestly so you can reason about them.

The payoff is scale without restructuring. A 3-person team can run all seven layers in a thin form on Monday morning. A 30-person team runs the same seven layers with more rows in each, more views slicing them, and more agents read-

ing them — but no new layers and no rearrangement. The Method is meant to survive headcount growth without a re-platforming exercise.

Initiatives — the objective layer

Definition. An initiative is a durable, quarter-or-longer outcome that groups projects under a single owner.

Initiatives are where the question “what are we trying to accomplish this quarter” gets a real, hyperlinked answer. They roll up projects (P-2), carry a target date, a health indicator, and an update cadence. They do not carry deadlines tighter than a quarter and they do not contain issues directly. The Method treats Linear as the active work graph (P-1), so an initiative without active projects under it is a smell — either revive it with a project or move the idea to a wiki.

For the 3-person team, the Operator’s first initiative might be **Platform Reliability** — a durable objective that survives every individual release. It rolls up two projects: a CI/CD Backbone the engineers are building now, and a Production Stability project that’s planned for next quarter. Health is “On Track.” Updates run biweekly (P-21). The Leadership Team consumes the initiative update; it does not browse the projects underneath.

Projects — the main execution container

Definition. A project is a bounded program of work with a single lead, a start and target date, milestones, and a weekly update.

Projects are the Method’s main execution container (P-3). Active work without a project is the second-most-common smell in a Linear workspace; the first is doc-shaped issues (covered under Issues). A project answers five questions explicitly: what is being built, by when, by whom, with what milestones, blocked by what. The Operator owns project shape; the project lead — often an engineer at this scale — owns the weekly status update (P-19).

Engineer A’s first project is **CI/CD Backbone**. Lead: Engineer A. Start: this week. Target: end of quarter. Milestones: Pipeline online → First service migrated → Documentation handed off. The project rolls up to the Platform Reliability initiative. It carries a Monday status update every week without exception (P-19), even when the update is “no movement, see last week.”

Issues — deliverable-sized work units

Definition. An issue is a single-outcome, single-owner work unit with exit criteria and an estimate measured in days, not weeks.

Issues are where execution actually happens. The Method’s rule is strict: one outcome, one owner, exit criteria you could read aloud at standup, and verification possible within two weeks (P-4). An issue whose acceptance criteria list more than five items, or whose implementation spans more than two days or two files, gets split into a parent issue with sub-issues (P-12). The Method also enforces a locality rule (P-18): an issue’s exit criteria override the project’s defaults, which override workspace defaults. Closer-to-the-work guidance wins.

Issues are also where the Method’s hardest discipline lives: documentation owns deep context (P-5). An issue is not the place for an ADR, a runbook, or a five-paragraph rationale. Link to the doc; keep the issue scoped to “what changed and how do we know.”

Engineer B’s first issue is **Add Linear-to-PR branch naming convention to the contributing guide**. Single outcome (the guide names the convention). Single owner (Engineer B). Exit criteria: PR merged, link posted to Engineer A and the Operator, two-line update in the parent project. Estimate: half a day.

Views — the operator’s daily control panel

Definition. A view is a saved filter over issues or projects that the Operator (and the team) checks on a recurring cadence.

Views are how the Method becomes operational. A workspace with no views is a workspace you can only navigate by clicking; a workspace with the right views is one where the Operator’s morning takes ten minutes. The Method requires a small standard set: a “Now” view scoped to in-flight work, a “Waiting on Operator” view, a “Needs Validation” view, a “Stale Projects” view, and a “Blocked Critical Path” view. Each view is read at a defined cadence — daily, weekly, or before reporting.

Views also justify the entire label system. Labels exist to serve views (P-16). A label that no view cites is a candidate for deletion. The Operator does not add a `domain:security` label because security feels important; they add it because the security-domain view is a real surface someone reads on a real day.

The 3-person team shares one Now view that filters to all issues in Ready, In Progress, In Review, or Validation. The Operator looks at it Monday morning. Engin-

eer A and Engineer B look at it before standup. It is the same view; the Method does not invent a separate operator view and a separate engineer view at this scale.

AUGMENTATION SURFACE · Layer Reconciliation

Where an agent layer makes this work better: scanning the workspace nightly for layer drift — an issue with three weeks of activity that should have been split into a project, a project whose only artifact is a long description and no issues that should have stayed a wiki page, a doc-shaped issue whose body would belong in a repo docs/ directory. The agent surfaces candidates with proposed new homes.

What the PM keeps: deciding the new home. Layer assignment is a judgment call about how the team will actually use the artifact next week, and that judgment stays with the Operator.

Intake — the entry point for new work

Definition. Intake is the set of surfaces through which new work enters the workspace: Triage, Asks (Slack/email), Customer Requests, and ad-hoc creation by the Operator or an agent.

Intake is the layer that fails silently when it fails. A team without an intake discipline accumulates work in three places: a Slack channel nobody triages, a doc nobody re-reads, and a Linear backlog of un-prioritized issues. The Method's rule is that Triage is the single funnel: all new work enters Triage and gets classified within seven days (P-20) — accepted into a project, merged with a duplicate, canceled, converted to a Customer Request, escalated to the Operator, or moved to docs-only. Customer feedback does not bypass Triage on its way to scope (P-22); it becomes a Customer Request, and the Operator decides if and when it converts.

Intake also includes the Operator's own quick captures and any agent-created issues. The Method treats Linear as the system of record (P-23) — capture in Linear, not in a side channel that requires later re-entry.

For the 3-person team, intake is thin: the Operator creates most issues, Engineer A and Engineer B drop the occasional “this should be tracked” into Triage, and a Slack-to-Asks intake gets installed when the team grows past five people. The discipline still applies: nothing sits in Triage longer than seven days unclassified.

Delivery evidence — the substrate for Validation

Definition. Delivery evidence is the set of artifacts produced outside Linear that prove an issue is implemented: PR URLs with merge status, CI results, deploy logs, screenshots, release notes, acceptance comments.

This is the layer that distinguishes the Method from teams that “use Linear.” Code that is merged is implemented; code that has evidence of running in the target environment is trusted (P-6). The Method’s Validation state (covered in detail in Chapter 10) is the gate between the two, and the gate is only as good as the evidence flowing into it. The code host produces that evidence (P-8): GitHub posts merge state, CI checks, and deploy outcomes; the GitLab equivalent posts merge request state, pipeline status, and deployment jobs. Linear links to those artifacts; Linear does not duplicate them.

For production-affecting work, no artifact means no Done (P-7). Engineer A’s CI/CD Backbone issue stays in Validation until the pipeline has run a real deployment end-to-end and the log is linked in the issue. The Operator’s job at that gate is to verify the link is real, not to take Engineer A’s word for it.

Analytics — the reporting substrate

Definition. Analytics is the layer of Insights, dashboards, and view-level metrics that converts the work graph into numbers the Operator and the Leadership Team can read.

Analytics is the last layer because it depends on every layer above it being clean. Throughput, cycle time, stale-project counts, Triage SLA health, agent throughput — all of these are derivable from a workspace where projects have updates, issues have evidence, and views are real. In a workspace where projects are stale and issues close without evidence, the same metrics measure theater.

The Method uses analytics sparingly at the 3-person stage: one Insights panel showing issue count by status on the Now view, one showing project age on the Stale view. As the team grows, dashboards consolidate the metrics the Operator hands upward — and the monthly review (Chapter 13) rolls initiative health up to the Leadership Team. The chain holds only because each lower layer carries its own evidence (P-28).

Layer-to-capability map

LAYER	PRIMARY LINEAR CAPABILITY	DEEP-DIVE CHAPTER
Initiatives	Initiatives + initiative updates	Chapter 13 (Reporting Up)
Projects	Projects, milestones, project updates, dependencies	Chapter 10 (Workflow Canon), Chapter 13
Issues	Issues, parent/sub-issues, exit criteria, labels	Chapter 5 (Operating Surface), Chapter 10
Views	Custom views, label filters, Insights panels	Chapter 5
Intake	Triage, Triage Intelligence, Asks, Customer Requests	Chapter 8 (Tier 3)
Delivery evidence	GitHub/GitLab integration, Releases, SLAs	Chapter 12 (Code Host Integration)
Analytics	Insights, dashboards	Chapter 13

Closing

The seven layers are necessary; you cannot drop one without weakening the work graph. The operating surface — labels, views, templates — covered in Chapter 5 is how those layers actually show up in the workspace day to day. The workflow canon and the Validation gate in Chapter 10 are the most opinionated part of the Method, and they sit on top of these seven layers. Everything that follows in this document assumes the layers are named, owned, and used as defined here.

WHERE THIS GETS HARD

Layer assignment becomes archeology when a team has historical work scattered across three shapes — some in tickets nobody updated, some in docs nobody linked, some in chat threads nobody can search — and the question “is this an issue, a project, or a wiki page” can’t be answered without reading a quarter of context for each item.

Ravenhelm assesses your operating surface and installs the Method tuned to your team, your code host, and your reporting structure. nate@ravenhelm.co

The Numbered Principles

You have been on three teams that all used Linear. One ran it like a public to-do list. One ran it like Jira, with seven custom states and a `Type` field nobody filled in. One ran it like a vibes-based Slack channel where Done meant somebody had stopped talking about it. None agreed on what a project was, what Done meant, or which artifact proved the work shipped — and none scaled past a dozen people without burning their PM out.

The Method does not solve that by adopting a tool harder. It writes down the decisions the Operator would otherwise re-derive every week and gives them numbers so they can be cited. The numbered principles are the spine — how the Method stays consistent across a 3-person team and a 30-person team, a Free workspace and a Business one, leadership that wants weekly slides and leadership that wants nothing.

The most load-bearing principle is **P-6 — Validation separates implemented from trusted**. If you apply one principle Monday, apply that one.

How to use the principles

The principles only work if they are *cited*. A principle never named in a PR comment, issue comment, project description, or agent scope brief is dead weight. A principle cited weekly becomes load-bearing — the team learns to argue at the rule, not the case.

Cite as `P-#` in PR comments (“Holding in Validation per P-7”), issue comments (“Splitting — see P-12”), project descriptions, agent scope briefs (“Per P-15, validation command is `pnpm test:e2e`”), and triage rejections (“docs-only per P-1”).

Cite frequently in the first month — the vocabulary calcifies fast, and once Engineer A writes `P-7` in a PR comment unprompted, the Method has installed it-

self. At six months, audit: a principle not cited and not visibly violated is either obvious enough to retire or so dormant the workspace is silently violating it.

AUGMENTATION SURFACE • Principle Citation Detection

Where an agent layer makes this work better: scanning issue and PR comments for P-# citations, tallying which are cited, which are dormant, and which appear in violation reports, and surfacing the citation graph as a monthly hygiene signal. The agent also flags completion claims that lack the prescribed evidence — closing without P-7’s artifact, or Done without P-9’s release evidence.

What the PM keeps: deciding whether a dormant principle should be retired, revived, or rewritten.

The 29 principles fall into ten groups (A-J). The counter-examples are diagnostic: when you see the failure in your workspace, you know which principle was quietly violated.

Group A — The work graph (the shape)

P-01 Linear is the active work graph, not an archive.

Rule. Content that does not represent active or imminent execution belongs in a wiki, a docs/ directory, or a project document — not in Linear.

Rationale. Linear is optimized for things that move. Static reference material corrodes the work graph by inflating lists, polluting search, and lowering signal; once a workspace tolerates archival content, every triage decision gets harder.

Counter-example. The Operator creates an “Onboarding checklist” issue with sixty items; it sits in Backlog nine months until archived, while three new hires print it from the team wiki.

CITED IN Ch 1 Ch 2 App. A

P-02 Initiatives are the objective layer.

Rule. Initiatives express durable outcomes that roll up projects, with no deadline tighter than a quarter.

Rationale. Projects ship and end; initiatives express the *why* that persists across multiple projects. Tying objectives to projects forces re-derivation every quarter; tying them to initiatives gives Leadership stable headings to recognize across reviews.

Counter-example. A team without an initiative layer rewrites OKRs from scratch each quarter; Leadership reads four framings of the same goal across four quarters and concludes the team has no strategy.

CITED IN [Ch 1](#) [Ch 2](#) [Ch 6](#) [Ch 7](#) [Ch 11](#) [Ch 13](#) [App. A](#)

P-03 Projects are the main execution container.

Rule. Issues belong to projects; a project answers what is being built, by when, by whom, with what milestones, and blocked by what — active work without a project is a smell.

Rationale. The project is the unit of accountability — where updates accrue, milestones gate progress, and Leadership reads health. An issue outside a project has no parent narrative and cannot be rolled up.

Counter-example. Engineer B on the 3-person team opens fifteen issues for “infrastructure cleanup” without a project; the monthly review shows zero progress because no status update captured any of it.

CITED IN [Ch 2](#) [Ch 5](#) [Ch 10](#) [Ch 11](#) [App. A](#)

P-04 Issues are deliverable-sized.

Rule. An issue has a single outcome, defined scope, exit criteria, and one owner; an issue that takes more than two weeks to verify should be split.

Rationale. Issues are the smallest unit the Operator and Leadership both reason about. Past two weeks they stop being deliverables and become workstreams — and workstreams are projects, with their own machinery.

Counter-example. A “Migrate the auth service” issue stays open three months; no update can say “60% done” because nothing in the work graph represents the 60%.

CITED IN [Ch 2](#) [Ch 4](#) [Ch 5](#) [Ch 6](#) [Ch 10](#) [Ch 11](#)

P-05 Documentation owns deep context.

Rule. ADRs, design docs, runbooks, and decision narratives live outside Linear, and Linear issues link to them rather than embed them.

Rationale. Linear’s issue descriptions are not version-controlled, diffable, or discoverable from outside the workspace. Long-form context written into an issue rots and cannot be cited by anyone without the URL.

Counter-example. Six months after launch the team cannot reconstruct why a service was built on Postgres instead of DynamoDB; the rationale lived in an issue description archived when the project closed.

CITED IN [Ch 2](#) [Ch 6](#)

Group B — Evidence and validation

P-06 Validation separates implemented from trusted.

Rule. Code that is merged is *implemented*; code that has passed runtime, browser, release, infrastructure, or human-acceptance evidence is *trusted* — only trusted code becomes Done.

Rationale. The single most load-bearing principle in the Method. Without it, the workspace collapses into a binary — merged or not — and every other principle is reduced to ceremony. Validation is where the Operator catches the gap between “the diff landed” and “the thing works in front of a user”; it is where production incidents are prevented and browser regressions get caught before Leadership notices. Teams that skip Validation discover within a quarter that their Done column is a lie. Validation costs throughput on the leaderboard and buys credibility everywhere else — that trade is the entire game.

Counter-example. Engineer A on the 3-person team merges a PR that “fixes the upload bug” and moves it straight to Done; at the weekly review Leadership asks if it was tested on Safari, the issue reopens Monday, and the team starts to fear status meetings.

CITED IN [Ch 1](#) [Ch 2](#) [Ch 4](#) [Ch 5](#) [Ch 6](#) [Ch 8](#) [Ch 9](#) [Ch 10](#) [Ch 11](#)
[Ch 12](#) [Ch 13](#) [Ch 14](#)

P-07 Evidence beats assertion.

Rule. Every claim of completion must point to an artifact — merged PR URL, deploy log, screenshot, document, or acceptance comment from someone other than the assignee.

Rationale. “I closed it” is not evidence. Once the team accepts the artifact discipline, Validation becomes self-enforcing — empty Done is Done that will be reopened.

Counter-example. Engineer A on the 3-person team closes seventeen issues Friday with no PR links; Monday the Operator cannot tell which shipped, and forensics confirms two did not.

CITED IN [Ch 2](#) [Ch 5](#) [Ch 6](#) [Ch 8](#) [Ch 10](#) [Ch 12](#) [Ch 13](#) [Ch 14](#)

P-08 The code host proves implementation.

Rule. Merge state, CI results, deploy logs, and runtime evidence live in the code host (GitHub, or the GitLab equivalent) — Linear links to them rather than duplicating them.

Rationale. Two sources of truth for implementation status drift instantly. Linear’s role is to reference what the code host says, not re-state it — re-statement is where lies enter (the issue says shipped, the deploy log says rolled back).

Counter-example. An Operator marks ten issues “deployed” from a Friday status meeting; the deploy rolled back overnight, but Linear still says shipped Monday when Leadership reads the update.

CITED IN Ch 2 Ch 5 Ch 6 Ch 8 Ch 10 Ch 12 Ch 13 Ch 14

P-09 Production work does not go Done before release evidence.

Rule. For customer-shipping issues, the Validation → Done transition requires release evidence — runtime smoke plus browser proof, or a deploy log plus an acceptance comment.

Rationale. Production-specific corollary of P-6. Internal-tool blast radius is the team; customer-facing blast radius is the brand, so the evidence bar is higher.

Counter-example. The team marks a customer-facing pricing change Done when the PR merges; three hours later a customer reports the price is wrong because nobody loaded the page after deploy.

CITED IN Ch 5 Ch 8 Ch 10 Ch 12 Ch 14

Group C — Workflow canon

P-10 The workflow canon is nine states.

Rule. The Method’s workflow is exactly nine states — Triage, Backlog, Ready, In Progress, In Review, Validation, Blocked, Done, Canceled — and adding states is a smell.

Rationale. Every additional state is a decision the Operator makes on every issue, every day. The 9-state canon is the smallest set that preserves the Validation distinction (P-6), the queue (Backlog vs Ready), and the obstruction (Blocked); workspaces with thirteen states are workspaces where the Operator has stopped reading the column.

Counter-example. A workspace adds “In QA,” “Pending Approval,” “Soft Launch,” and “Awaiting Release” over six months; no two engineers agree on In Review, and Validation sits empty because work routes around it.

CITED IN Ch 5 Ch 6 Ch 9 Ch 10 Ch 11 App. A

P-11 Stage labels carry the things that look like states but aren’t.

Rule. “In QA,” “Pending Approval,” “Design Review,” and similar are not workflow states — they are `stage:*` labels applied during In Progress, In Review, or Validation.

Rationale. Sub-stages matter for filtering; the answer is a label, not a state. Labels can be added without breaking the workflow contract, view-filtered without breaking analytics, and team-specific without fragmenting the workspace.

Counter-example. A team adds “In Design Review” as a state, then has to redefine cycle-time analytics, retrain every Insights view, and write a migration script — a `stage:design-review` label would have done the same in five minutes.

CITED IN Ch 5 Ch 6 Ch 10 App. A

P-12 Prefer parent issues with sub-issues over giant single issues.

Rule. If acceptance criteria exceed five items, or implementation spans more than two files or two days, split into a parent issue with sub-issues.

Rationale. Giant issues defeat estimation, parallelization, and the evidence chain. A parent-and-children structure preserves the narrative while making each unit independently reviewable, assignable, and verifiable.

Counter-example. Engineer B on the 3-person team takes a single issue titled “Implement billing v2”; three weeks later the Operator cannot answer “what’s left?” because nine acceptance criteria are not discrete units.

CITED IN Ch 2 Ch 5 Ch 6 Ch 10

Group D — The Operator

P-13 The Operator owns priority, taste, risk, and real-world commitments.

Rule. Priority, taste, risk judgment, and real-world commitments do not delegate — an agent or an Engineer can propose, surface, or draft them, but the Operator decides.

Rationale. These four are where being wrong costs trust, money, or outside relationships. An agent can *surface* a drift or *draft* a commitment, but the moment those decisions happen without the Operator’s acknowledgment, the team loses the human accountability that makes the structure legible upward.

Counter-example. The Operator on the 3-person team lets an agent auto-promote issues from Low to Urgent on customer-mention frequency; by Friday Engineer A has been reassigned to four “urgent” tickets nobody committed to, and a leadership-promised demo slips two days.

CITED IN Ch 1 Ch 4 Ch 5 Ch 6 Ch 8 Ch 11 Ch 14 App. A

P-14 Agents are executors, not owners.

Rule. Specialist agents can be delegated work, but human ownership stays explicit on every issue via the assignee field — even when a `delegate` field is set to a bot.

Rationale. The assignee is the throat to choke; setting the assignee to a bot makes the issue ownerless in any meaningful human sense. The split — assignee=human, delegate=bot — keeps agent productivity while preserving accountability. As agents move from drafting comments to running coding sessions that open pull requests, the delegate can now change the codebase directly — which raises the cost of an ownerless issue, not lowers it. The human assignee owns what the agent ships.

Counter-example. A workspace assigns issues to an AI bot user; six weeks later the bot has “owned” thirty issues, two regressed silently, with no human to ask why.

CITED IN `Ch 4` `Ch 8` `Ch 9`

P-15 Agent delegation requires acceptance criteria and a validation command.

Rule. Setting a delegate on an issue requires a scope comment naming outcome, scope, validation command(s), expected artifact, and repo or file paths.

Rationale. Agents fail on under-specified inputs in ways that look superficially fine. Without a validation command, the Operator can only audit by reading the diff line by line — exactly the work delegation was supposed to save. This is sharper for coding-session agents that open pull requests: the validation command is the acceptance test the Operator runs instead of re-reading generated code.

Counter-example. The Operator delegates “fix the flaky tests” with no validation command; the agent disables three of them, and the Operator does not notice until a regression ships two weeks later.

CITED IN `Ch 4` `Ch 5` `Ch 8` `Ch 9` `Ch 14`

Group E — The operating surface

P-16 Labels exist to serve views.

Rule. A label not cited by any view is a candidate for deletion — labels are an index, not an organizational scheme.

Rationale. Every label is a small tax on every issue creation. The “serves views” test keeps the namespace lean and the Operator’s filter vocabulary small enough to remember.

Counter-example. A workspace has 240 labels; an audit finds 180 referenced by zero views, created during a single project four quarters ago.

CITED IN [Ch 1](#) [Ch 2](#) [Ch 5](#) [Ch 6](#) [Ch 9](#) [App. A](#)

P-17 Templates lower the cost of doing the right thing.

Rule. Every recurring work type — bug, feature, research spike, production incident, agent delegation — gets an issue template.

Rationale. A blank issue invites skipping the structure. A templated issue nudges the creator toward the format the Operator and the validation gate both expect, encoding the Method without anyone having to remember it.

Counter-example. The team has no production-incident template; the next outage is recorded as a free-form issue with no severity, no commander, no runtime-evidence link, and the post-mortem takes three times as long.

CITED IN [Ch 5](#) [Ch 6](#) [Ch 8](#) [Ch 9](#) [App. A](#)

P-18 Locality wins.

Rule. Closer-to-the-work guidance is more authoritative — a repo-level convention overrides workspace defaults for that repo, a project description overrides workspace defaults for that project, and an issue’s acceptance criteria override project defaults for that issue.

Rationale. Global rules are baselines and are wrong in any sufficiently specific context. Locality lets the Method scale across many projects without forcing every project into the same shape — the auth repo can be stricter than the workspace default, and a one-off migration can override the project’s normal Done.

Counter-example. A workspace globally mandates “every PR must include a benchmark”; a one-line CSS bugfix sits open three days because the engineer is inventing a benchmark for a color change.

CITED IN Ch 2 Ch 5 Ch 7 Ch 8 Ch 9 Ch 12

Group F — Cadence**P-19 Active projects get a status update every week.**

Rule. A project with no update for seven days surfaces in a stale-projects view; the cadence is non-negotiable, though the format compresses when there’s nothing to say.

Rationale. Project updates are the rolling truth-telling that prevents the monthly review from becoming a surprise. The compressible format (“Still in progress, no risks”) is the release valve.

Counter-example. A project goes four weeks without an update during a steady stretch; in week five it goes At Risk and Leadership is blindsided because the last status they have is from a month ago.

CITED IN Ch 2 Ch 4 Ch 5 Ch 6 Ch 9 Ch 11 Ch 13 App. A

P-20 Triage has a seven-day SLA.

Rule. By day seven, an item in Triage is classified — accepted, merged, canceled, converted to a customer request, escalated to the Operator, or moved to docs-only.

Rationale. Triage is a queue, not a holding pen. Items that sit there become the workspace’s unread inbox; the seven-day clock forces a decision even if it’s “this is documentation.”

Counter-example. Triage swells to ninety items over a quarter; the Operator stops opening it, and a Po customer report sits nine days before anyone notices.

CITED IN Ch 2 Ch 5 Ch 8 App. A

P-21 Initiative updates run biweekly; the monthly review rolls initiatives up to Leadership.

Rule. Project updates feed initiative updates; initiative updates feed the monthly review; each layer compresses the one below it, and Leadership consumes the top of the stack.

Rationale. The cadence pyramid is the Method’s reporting structure. Weekly suits the team; biweekly compresses enough to write without re-doing work; monthly fits an executive audience.

Counter-example. A team writes weekly updates but never compresses them; Leadership receives forty-seven raw updates in a month and either reads none or reads them all badly.

CITED IN Ch 2 Ch 4 Ch 11 Ch 13 App. A

Group G — Intake and feedback

P-22 Customer feedback does not directly become scope.

Rule. A customer request is captured and triaged; conversion to a project or issue is an Operator decision (or, once trust is established, an agent-layer decision routed through a Waiting-on-Operator view).

Rationale. Customer feedback is signal, not commitment. Direct conversion is how roadmaps get hijacked by the loudest customer; Triage is where the Operator weighs the request and accepts, parks, or declines.

Counter-example. The team installs a feedback widget that opens issues directly into Backlog; six weeks later the backlog is half customer suggestions and the Operator has stopped reading it.

CITED IN [Ch 2](#) [Ch 6](#) [Ch 7](#) [Ch 8](#) [Ch 9](#) [Ch 14](#) [App. A](#)

P-23 The system of record is Linear.

Rule. Two-way sync with external trackers is debt — webhooks in are fine, but bidirectional sync is not.

Rationale. Every two-way sync has drift: updates that did not propagate, statuses that overwrote deliberate edits, deletions that left orphans. Single-source-of-truth is harder upfront, dramatically easier downstream.

Counter-example. A team installs bidirectional sync between Linear and another ticket system; three months in, the Operator spends a Friday figuring out why an issue is Done in one and Open in the other, and concludes neither can be trusted.

CITED IN [Ch 2](#) [Ch 9](#) [Ch 12](#)

Group H — System hygiene

P-24 The overbuild test.

Rule. Before adopting any new feature, label, view, or template, answer five yes/no questions: does it help the Operator decide faster, does it help an executor execute faster, does it preserve evidence we'd otherwise lose, does it reduce repeated PM work, does it make risk visible earlier — four or more yes, adopt; three or fewer, skip.

Rationale. Workspace bloat is the failure mode of every PM who likes their tool. The five questions force the Operator to articulate what the new thing buys.

Counter-example. The Operator on the 3-person team adds a view “Issues Tagged Frontend” because it seemed useful; six months later it has been opened twice and nobody remembers it.

CITED IN [Ch 5](#) [Ch 6](#) [Ch 7](#) [Ch 8](#) [Ch 9](#) [Ch 11](#) [Ch 12](#) [Ch 14](#) [App. A](#)

P-25 Verify what you enforce.

Rule. Don't enforce a convention you can't check programmatically.

Rationale. Manual enforcement scales to one person on one day; programmatic enforcement scales to every PR and every issue without the Operator being the bad cop. If the rule cannot be automated, find a cheaper enforcement point or accept the rule will erode.

Counter-example. A workspace adopts “every commit message must include the Linear issue ID”; with no pre-commit hook, compliance drifts from 80% in month one to 30% by month four, and the dependent integration rots.

CITED IN [Ch 5](#) [Ch 8](#) [Ch 9](#) [Ch 12](#) [Ch 13](#) [App. A](#)

P-26 Capacity commits to seventy percent.

Rule. Sprint or cycle planning commits seventy percent of expected capacity; the remaining thirty percent is reserved for unplanned work. Cycles are optional in the Method (Chapter 11); the seventy-percent rule applies to whatever planning horizon the team commits against, whether a cycle, a milestone, or a week.

Rationale. Unplanned work is recurring, not a special case. Teams that commit to 100% slip every cycle and learn nothing because the explanation is always “something came up”; teams at 70% finish on time and surface real overcapacity when they finish early.

Counter-example. Engineer A and Engineer B each commit to ten points; an incident takes a day and an urgent customer call takes a half-day, and by Friday both have shipped six and the Operator apologizes to Leadership for the third week running.

CITED IN Ch 11 Ch 14

Group I — Reporting

P-27 Leadership consumes summaries, not raw Linear.

Rule. Leadership reads project updates and initiative health — they do not browse the workspace.

Rationale. Raw Linear is an engineering surface. Leadership reads it the way an executive reads a database schema: technically possible, practically wrong. The Operator’s translation is the value-add.

Counter-example. The Leadership Team on the 3-person team asks for “real-time visibility” and gets a board link; they log in once, find it overwhelming, and at the next standup ask the Operator a question any project update would have answered — but the Operator stopped writing them because Leadership “had the board now.”

CITED IN Ch 1 Ch 4 Ch 6 Ch 8 Ch 9 Ch 11 Ch 13 Ch 14 App. A

P-28 The evidence chain holds.

Rule. Initiative health rolls up from project updates, project updates roll up from issue evidence, and issue evidence rolls up from the code host — break any link and the chain collapses into theater.

Rationale. Every rollup is only as honest as the layer underneath. Initiative health that ignores project updates is a vibe; project updates that ignore issue evidence are a guess; issue evidence that ignores the code host is a story. With the chain intact, the monthly review is defensible all the way to the merge commit.

Counter-example. A monthly initiative report says On Track; project updates have not been written in three weeks, issues have no evidence links, and the Operator cannot trace a single claim to an artifact.

CITED IN [Ch 1](#) [Ch 2](#) [Ch 6](#) [Ch 9](#) [Ch 10](#) [Ch 11](#) [Ch 13](#) [Ch 14](#)
[App. A](#)

Group J — Releases

P-29 Releases are named with Semantic Versioning.

Rule. Every release carries a MAJOR.MINOR.PATCH version — MAJOR for a breaking change, MINOR for a backward-compatible feature, PATCH for a fix. The number is derived from Conventional Commits by `semantic-release` (or `release-please`), not chosen by hand.

Rationale. A release surface that grows without a naming law becomes unanswerable: “which version shipped the auth fix?” has no reliable answer when the tags are `v2`, `final`, `hotfix-2`, and `july-build`. SemVer makes the version, the changelog, and the tag a byproduct of commit discipline rather than a manual decision, so the release history stays legible as it grows — and the code host, not a human, keeps it honest (P-25).

Counter-example. A team tags releases by hand — `1.0`, then `1.0-real`, then `1.0-real-final`, then `prod-2026-06` — and three months later cannot tell which deploy carried which fix; a customer-reported regression takes a day to bisect because no version maps to a known change set.

CITED IN [Ch 12](#) [App. A](#)

Closing

Principles are not laws. They are decisions made in advance, written down with numbers, so the Operator does not have to decide them again at 4pm Friday when an issue sits in Validation and Leadership asks why it isn't shipped. The Friday answer is already written. The Operator remembers the number and cites it; the Method makes the number worth remembering.

WHERE THIS GETS HARD

The moment Leadership disagrees with a load-bearing principle — most commonly P-6, because they want Done to mean merged, not validated — the Method has to be defended in a calendar meeting against a stakeholder who outranks the Operator. The defense is not technical; it is about which incidents the team is willing to ship into next quarter, and most Operators do not have the seniority or the artifact library to make that argument alone.

Ravenhelm assesses your operating surface and installs the Method tuned to your team, your code host, and your reporting structure. nate@ravenhelm.co

The PM Operator Role

INSTALL PATH

PRINCIPLES CITED

P-4

P-6

P-13

P-14

P-15

P-19

P-21

P-27

It is Wednesday afternoon. The Operator has been in Slack since 8 a.m., answering pings that each take ninety seconds and collectively erase the day. Engineer A is waiting on a yes-or-no decision the Operator answered last week but cannot find. Engineer B has opened a pull request that touches three repos and is asking whether it can ship before a customer call on Friday. The Leadership Team has asked, again, for “a quick status on where we are.” The Operator has not opened Linear since lunch. There is no time to think about priority because there is no time that is not already spoken for.

This is the failure mode the Method is built to fix. Not by adding more process. By making it crisp what the human PM — the Operator — owns, what they hand off, and what shape their week takes when the system is doing its job. Everything else in the Method exists to give the Operator back the room to do the four things only they can do.

What the Operator owns

The Operator owns four things and cannot delegate them (P-13). An agent can draft, a teammate can recommend, Leadership can push — but the decision is the Operator’s. The four are priority, taste, risk, and real-world commitments.

Priority. What is first, what is next, what is deferred. Priority looks like a list, but it is really a sequence of trade-offs: this customer regression before that platform refactor; the security patch before the demo polish; the planning spike before another sprint of building blind. Priority cannot be delegated because the cost of getting it wrong is paid by the team that has to live with the choice. Agents can rank backlogs, surface stale work, and propose orderings (P-14). The Operator decides which proposal becomes the plan.

Taste. What “good enough” looks like for this team, this product, this customer. Taste is the call that an engineer’s PR is ready to merge even though it does not handle the exotic edge case; or that the dashboard’s copy needs another pass before Leadership sees it; or that the integration is fine in staging but not yet in front of the paying account. Taste is not subjective in the unhelpful sense — it is calibrated to context. Outsiders cannot make it. New hires take months to acquire it. The Operator carries it for the team.

Risk. Security exposure, compliance posture, blast radius, what the team can afford to ship without breaking trust. Risk shows up as the question behind a decision: if this PR goes wrong, what breaks, and for whom? The Operator does not need to be a security engineer to own risk — they need to be the person who refuses to let a risky change ship without naming who has the authority to accept the risk. Agents can surface risk signals (failed checks, missing tests, unfamiliar dependencies). The Operator decides the team’s tolerance.

Real-world commitments. Promises made to customers, leadership, partners, regulators. The 10 a.m. demo on Tuesday. The renewal owed by month-end. The compliance attestation due before audit. These do not delegate because their consequences do not delegate; the person whose name is on the calendar is the person who absorbs the fallout. The Operator’s job is to keep these commitments visible inside the work graph so they do not collide silently with execution.

These four are not a job description. They are a load-bearing definition. Strip them away and you have a clerk. Hold them and you have an Operator.

The PM Operator practice ladder

Most PMs reading this are not at the Operator level yet. They are climbing toward it from below, or sliding into it from above and discovering they no longer have the proximity to the work that the role demands. The Method targets the Operator rung specifically. The ladder below describes what each rung looks like in practice and how to tell where you stand.

LEVEL	WHAT THEY DO	SELF-ASSESSMENT CUE
Clerk	Keeps tickets clean. Mirrors decisions made elsewhere. Updates status fields after the fact.	“I am writing down what already happened.”
Coordinator	Runs the rhythm — weekly updates, triage, status reports. Priority comes from above.	“I run the meetings. Someone else decides what we work on.”

LEVEL	WHAT THEY DO	SELF-ASSESSMENT CUE
Operator	Owens priority, taste, risk, and commitments inside their team’s scope. The Method targets this level.	“If I disappeared for a week, the team would not know what to ship next.”
Program Manager	Runs multiple Operators across initiatives. Coordinates across teams without losing the four.	“I am responsible for outcomes I cannot personally see being built.”
Chief of Staff	Sets the operating model itself. Decides what the Operators are accountable for.	“I am writing the rules other PMs operate under.”

A Clerk who installs the Method does not become an Operator overnight; they become a Clerk with better tooling. The promotion happens when the four start landing in their lap and they hold them instead of escalating them. A Coordinator becomes an Operator the first week they say “no, not this sprint” and make it stick. The Method is the structural support that makes that transition survivable — clear ownership of the four (P-13), explicit acceptance criteria for everything delegated (P-15), a weekly cadence that produces real visibility (P-19, P-21) — but it does not perform the transition. Only the human does.

If you read these rungs and recognize yourself at Clerk or Coordinator and want to climb to Operator: that is the entire reader profile of this document. Keep going.

The daily and weekly shape

The Operator’s week has a deliberate shape. Without one, the role collapses back into reactive triage and the four get squeezed out by whatever pinged loudest. With one, the four get the time they need and the rest of the work runs on rails.

Monday morning. The Operator opens the workspace, reviews every active project, and drafts that week’s project updates. Priorities for the week are confirmed — or revised — with Engineer A and Engineer B in a thirty-minute standup. The output of Monday is a workspace where every active project has a current update (P-19), every engineer knows what they own this week, and the Operator knows which two or three decisions they will need to make before Friday.

Daily. The Operator walks four views in order: Triage (anything new since yesterday), Blocked (anything that has stopped moving and why), Waiting on Operator (decisions queued for them specifically), and the agent-delegation surface (work handed off, with its return state). This is a fifteen-minute pass, not a meeting. Items that need a decision get one. Items that need scope get scoped. Items

that are stuck because someone is waiting on the Operator get unstuck because the Operator is, in fact, here.

Friday afternoon. The Operator posts a Friday summary covering five things: what shipped, what is still open, what is blocked and why, what decisions are pending, and what the next week’s focus is. This is the artifact Leadership reads (P-27). It is also the artifact the Operator reads next Monday when they ask themselves what they thought the team was doing.

After any PR merge. The Operator (or the system on their behalf) checks whether the issue needs Validation evidence before it can move to Done. Merged is not done (P-6). A PR that closes an issue automatically without runtime, browser, or release proof is a Done state the team cannot trust — and trust is the whole reason the workflow has a Validation gate at all.

AUGMENTATION SURFACE • Operator Extension

Where an agent layer makes this work better: drafting the Friday summary from project updates, classifying Triage items by domain and risk, detecting projects whose status has gone stale, summarizing a week of issue activity into a paragraph Leadership will actually read, and proposing first-pass priority orderings from the current backlog. All of this is work that *looks like* priority but is actually translation — moving information between layers without changing what it means.

What the PM keeps: every priority decision, every “good enough” call, every commitment to a customer or partner, and every accept/reject on what the agent layer proposes. The agent extends the Operator’s reach; it does not change what the Operator owns.

Mapping upward and downward

The Operator sits between two audiences with different needs. The role is, in large part, the work of translating between them.

Upward, to the Leadership Team. Leadership consumes summaries, not raw Linear (P-27). The Operator translates project activity into project updates, project updates into initiative health, and initiative health into the monthly review that Leadership uses to decide what the company funds next. The Operator does not let Leadership browse the workspace. Browsing produces noise — half-written issues, abandoned spikes, internal arguments in comment threads — and noise produces bad questions, which produce bad meetings, which steal the Operator’s week. The contract upward is: you get a current, honest summary every

week (P-19); you trust the summary; if you want to drill in, you ask the Operator and they will pull the thread.

Downward, to Engineer A and Engineer B. The Operator clears the path. Every issue handed to an engineer has acceptance criteria explicit enough that the engineer can ship without coming back for clarification (P-4). Every delegation — to a person or to an agent layer — carries scope, exit criteria, and a way to validate the result (P-15). The Operator does not stand over engineers' shoulders; they front-load clarity so they do not need to. When Engineer A picks up a Ready issue on Tuesday morning, the issue should answer: what is the outcome, what is in scope, what is not, what artifact proves it is done, and what repo or file path is the work in. If any of those are missing, the Operator owes the engineer five minutes of work *before* the engineer starts, not five interruptions during.

The two examples that make this concrete: priority is the Operator deciding on Monday that this week the Platform Reliability project comes before the Internal Tooling Integration project, and saying so out loud in the standup. Execution is Engineer B picking up the top issue in Platform Reliability and shipping it by Thursday without needing the Operator to weigh in on any sub-decision along the way. The Method's whole shape is in the gap between those two sentences — one stays with the Operator, one gets handed off, and the handoff is clean because the four (P-13) have been done in advance.

The Operator is not a status-reporting machine. They are the load-bearing decision-maker between Leadership's strategy and Engineering's execution — the human who owns priority, taste, risk, and commitments, and who keeps the work graph honest so the rest of the team can move.

WHERE THIS GETS HARD

The role starts to crack when the team grows past five engineers — the Operator can no longer hold the full backlog in their head, and the four (P-13) start getting silently delegated to whoever ships fastest. It also cracks when the Operator gets pulled into a leadership-level decision that requires Chief-of-Staff-shaped work — cross-team trade-offs, budget reallocation, org design — that the Method, by design, has not installed yet.

Ravenhelm assesses your operating surface and installs the Method tuned to your team, your code host, and your reporting structure. nate@ravenhelm.co

The Operating Surface: Labels, Views, Templates

INSTALL PATH	PRINCIPLES CITED	P-3	P-4	P-6	P-7	P-8	P-9	P-10	P-11	P-12
P-13	P-15	P-16	P-17	P-18	P-19	P-20	P-24	P-25		

Open a workspace that has been running without a method for a year. Three engineers each invented their own label scheme, so the workspace has `bug`, `Bug`, `type:Bug`, and `defect` — all of them in use, none of them filtered against. Half the labels are attached to a single issue. The views panel shows nineteen private views and three shared ones, and the three shared ones don’t agree on what “active” means. The Operator opens Linear in the morning, sees a different workspace than either engineer sees, and spends the first forty minutes reconstructing what changed overnight. There is no single screen anyone can point to and say *this is what we are working on now*.

That is the gap this chapter closes. The seven layers in Chapter 2 describe the shape of the work graph. The numbered principles in Chapter 3 describe the doctrine. This chapter is how the Method actually shows up in the workspace — the labels you index against, the views the team watches, and the templates that make it cheaper to do the right thing than the wrong thing.

The operating surface is the *enforceable* part of the Method. Principles in a document live in heads. Labels and views live in the workspace. When two team members disagree about whether a piece of work is in flight, the answer is in a saved view — not in conversation. That makes the operating surface the verification surface (P-25): a rule you can’t check programmatically decays into folklore. Labels and views are the check.

Labels are an index, not an organizational scheme

The single most common mistake in a one-year-old workspace is treating labels as a filing cabinet. The Method treats them the opposite way: a label exists to serve a view, and a label not cited by any view is a candidate for deletion (P-16). That inverts the usual instinct. You do not create a `frontend` label because work is frontend. You create a `domain:product` label because a specific view filters on it and the Operator needs that view to decide something.

The Method uses eight namespaced label groups. They are namespaced because flat label spaces collapse to noise inside six months, and because the colon prefix lets you scan a label list and see structure instead of an alphabetical pile.

- `type:*` — Feature, Bug, Chore, Documentation, Research, Gate. Every issue gets exactly one. The “Gate” type marks issues whose only purpose is to hold a decision or an approval until it lands.
- `component:*` — one per service or subsystem the team owns. Let the reader fill in the names; the Method does not prescribe them. A team running three services might have `component:api`, `component:web`, `component:worker`. A team with one monolith might skip this namespace until they split.
- `domain:*` — broad areas of concern: `domain:security`, `domain:infrastructure`, `domain:product`, `domain:operations`. Use sparingly. Domain answers “who would be interested in this if it broke” — not “what code does it touch.”
- `risk:*` — `low`, `medium`, `high`. Applied at Triage. Drives one view (Blocked Critical Path) and one report (the weekly risk roll-up in Chapter 13).
- `source:*` — `asks`, `slack`, `support`, `github`, `sentry`. Where the work came in from. Drives Triage analytics: if eighty percent of incoming work is `source:slack`, the team needs Asks (Chapter 7), not more meetings.
- `release:*` — `production`, `staging`, `internal`, `none`. Marks the deployment surface so the Validation gate (Chapter 10) knows what evidence to require. `release:production` means runtime evidence is non-negotiable before Done (P-9).
- `stage:*` — `design`, `qa`, `uat`, `soft-launch`. These are the “states that aren’t states” (P-11). The workflow canon is nine states (Chapter 10); anything that looks like a tenth state — pending approval, in QA, soft-launch — is a stage label applied during In Progress, In Review, or Validation. The canon stays at nine.
- `parking-reason:*` — `waiting-on-vendor`, `waiting-on-decision`, `waiting-on-budget`, `waiting-on-customer`. Applied only to Urgent or High issues sitting in

Backlog. An Urgent issue in Backlog without a parking reason is a smell — either the priority is wrong or the reason is hiding.

Each namespace serves at least one view. `type:Bug` plus `risk:high` plus `release:production` is the filter behind the Operator’s morning incident scan. `source:slack` is the filter behind the weekly Asks review. `stage:uat` is the filter behind the Validation queue. `parking-reason:*` populates a sanity-check view the Operator runs before standup: *what am I claiming is urgent that I am actually not working on, and why?*

This is P-16 in action. No view, no label. Before adding `type:Spike`, ask which view it serves. If the answer is “I just want to find them later,” that is what search is for.

One more discipline before views: prefer Linear’s native typed fields over labels for anything Linear already models. Priority, estimate, workflow state, and project health are single-valued typed fields — a label that duplicates one of them (a `priority:high` label, a red health label, a done label) is drift waiting to happen, because a label is set membership and a native field is a single enforced value. The `bug/Bug/defect` collision that opens this chapter is what labels do when they impersonate a typed field. Reach for a label only when the concept is a genuine many-to-one index that no native field captures.

AUGMENTATION SURFACE · Surface Maintenance

Where an agent layer makes this work better: detecting labels with zero referencing views, views with no recent activity, templates that no one uses, and stage labels that have outlived their underlying state. The agent generates a monthly hygiene report — *these 12 labels are referenced by no view; these 4 views were last opened three months ago; this template has been used twice in 90 days* — so the Operator can make deletion decisions on evidence instead of guesswork.

What the PM keeps: deciding whether a dormant artifact should be removed, repurposed, or left in place because a future workflow needs it.

Views are the daily control panel

A view is a saved question. The Method requires a small set of shared views — shared so every team member sees the same workspace — and treats them as the daily control panel. The Operator opens these views before standup; the engineers open the ones that touch them. Private views are fine for personal scratch work, but the load-bearing views are shared.

Operator's Now. Filter: priority Urgent or High, state in Ready, In Progress, In Review. Group by assignee. This is what is actively in flight. The Operator watches it every morning. Engineer A and Engineer B each glance at it before picking up new work. It enforces **P-3** (active work has a project) by making project-less issues look out of place, and **P-4** (issues are deliverable-sized) by making oversized issues conspicuous.

Waiting on the Operator. Filter: issues whose state is not Done and whose human-decision flag is set, plus issues whose blocker text names the Operator. Only the Operator subscribes; engineers see it on request. This is the queue **P-13** makes load-bearing — priority, taste, risk, and real-world commitments do not delegate. If this view has more than five items, the Operator's calendar is wrong, not the engineers' execution.

Blocked Critical Path. Filter: state Blocked, priority Urgent or High. Engineer B owns this view because they sit downstream of the most platform dependencies and are blocked most often; they raise items in standup. It enforces **P-10** — Blocked is a state of the canon, and an issue in Blocked must name its blocker. A Blocked issue with no named blocker is a triage failure.

In Review. Filter: state In Review with at least one linked PR. The view shows the PR status (checks passing, requested reviewers, mergeable). Both engineers subscribe; the Operator scans it once a day to spot review backlog. This is where **P-8** lives: the code host proves implementation, and this view is the bridge between Linear and that proof.

Needs Validation / Evidence. Filter: state Validation, age greater than 24 hours. The Operator and Engineer A both subscribe. This view is the Validation gate (Chapter 10 owns the doctrine; this chapter owns the surface). An item sits here until evidence is posted — a deploy log, a screenshot, an acceptance comment from someone other than the assignee (**P-7**). Validation separates implemented from trusted (**P-6**), and this view is where the separation gets enforced day-to-day.

Projects Without Recent Updates. Filter: project status Active, last update older than seven days. The Operator watches it on Monday morning. This view operationalizes **P-19** — active projects get an update every week, the format can compress when there is nothing to say, the cadence does not move. If a project surfaces here, the Operator either writes the update or moves the project to Planned.

Stale Triage. Filter: state Triage, age greater than five days. Surfaces **P-20** violations one to two days before the seven-day SLA so the Operator has runway, not just an alarm. Operator-only.

Seven shared views, named for the question each one answers. The fight is to keep this list short. Every new view dilutes the daily scan — when there are eighteen shared views, no one looks at any of them. When there are seven, the team agrees on what they are looking at.

Templates lower the cost of doing the right thing

A template is a default. The right behavior should be the easy behavior (**P-17**). The Method requires four issue templates and one project template at minimum, plus a commenting template for agent delegation that becomes load-bearing at Tier 3 (Chapter 8).

Feature template. Sections: Outcome (one sentence — what changes for whom), Scope (in/out, plus what is explicitly deferred), Acceptance criteria (testable bullets, not prose), Evidence expected (which artifact proves Done — deploy log, screenshot, acceptance comment), GitHub repo (the GitLab equivalent is the GitLab project path), Design link (when visual review applies). The template forces the issue author to decide what Done looks like before opening the issue, which is the cheapest moment to decide.

Bug template. Sections: Reproduction (exact steps, environment, version), Blast radius (who is affected and how many), Severity (P0–P3 with the Method’s definitions inline so the author cannot guess), Evidence required for the fix (regression test, manual repro confirmation, monitoring confirmation). The blast-radius field is the one most teams skip and the one that most often determines whether a bug deserves the priority it was opened with.

Research spike template. Sections: Timebox (hours or days — a spike with no timebox is a project in disguise), Question the spike answers, Expected artifact (a decision doc, a prototype branch, a benchmark table), Decision deadline (the date by which the spike’s output must drive a decision or get archived). Spikes that produce no decision artifact are how research budgets disappear.

Standard project template. Sections: Goal (the durable outcome, not the next step), Milestones (the Method’s canonical phases — Scope accepted, Implementation, Validation, Production rollout, Documentation and handoff — plus any project-specific phases), Dependencies (other projects, vendor commitments, hiring), First update (drafted at kickoff, posted within forty-eight hours), Exit criterion

ia (what makes this project Done, in plain language). Every project starts from this template. A project without a first update within forty-eight hours surfaces on Projects Without Recent Updates by day seven.

Agent delegation comment template. A comment, not an issue. Used when the Operator (or, at Tier 3, the agent layer) hands an issue to a specialist agent. Sections: Outcome, Scope, Validation command (the exact command or check that proves the work is correct), Expected artifact (PR URL, document, file path), Repo and file paths. Required by P-15 — vague delegation produces vague output. Chapter 8 carries the full pattern; this chapter installs the comment template so the muscle is in place before agents arrive.

These templates encode locality (P-18). The template is the workspace default; a project description can override it for that project; an individual issue's acceptance criteria override the project default for that issue. Closer-to-the-work guidance wins.

Relations and recurring issues

Two native primitives the Method uses deliberately but does not over-invest in.

Issue relations. Linear models relations between issues — *blocking* / *blocked-by*, *related*, and *duplicate*. The Method leans on the dependency pair and uses the rest sparingly. *Blocked-by* is the structural complement to the Blocked state (P-10): the state says “this is stuck,” the relation says “on what.” When a Blocked issue is held up by another tracked issue, link it *blocked-by* in addition to naming the blocker in a comment — so the dependency shows on the blocker's side too and nobody has to carry the chain in their head. *Duplicate* is set at Triage (it is a native status type). *Related* is for genuine cross-references, not a substitute for a parent issue (P-12) or a project — if you are building a tree out of *related* links, the work wanted a parent-and-sub-issues shape instead. Relations are an index of dependency, not a second hierarchy.

Recurring issues. Repeating operating work — the weekly project-update sweep, the Monday Triage clear, the monthly label-hygiene pass, the quarterly over-build audit (P-24) — should be a recurring issue, not a reminder in someone's head or a line in a doc that rots. A recurring issue puts the cadence in the work graph where the Method can see it: it appears in views, carries evidence when it is done, and surfaces on Stale Triage if it is ignored. The Method's own maintenance — the verification it depends on (P-25) — is the first thing to make recurring.

The overbuild test

Before adding any new label, view, or template, run it through P-24's five questions. Does it help the Operator decide faster? Does it help an executor execute faster? Does it preserve evidence the team would otherwise lose? Does it reduce repeated PM work? Does it make risk visible earlier? Four or more yes — adopt. Three or fewer — skip.

The operating surface gets cluttered the same way a kitchen counter does: each individual item seemed useful when it landed, and now nothing can be found. The overbuild test is the cabinet door. Apply it every time and the surface stays operational. Skip it for a year and the surface becomes the problem the next operating model is brought in to fix.

WHERE THIS GETS HARD

When label sprawl has gone unmanaged for a year and an audit reveals one hundred fifty labels of which thirty are cited by views, the deletion sweep is political, not technical — every label has someone who created it and a story for why it should stay. The same dynamic hits a view list that has accumulated nineteen private dashboards no one else can see.

Ravenhelm assesses your operating surface and installs the Method tuned to your team, your code host, and your reporting structure. nate@ravenhelm.co

Tier 1: Linear Free Install

INSTALL PATH	PRINCIPLES CITED	P-2	P-4	P-5	P-6	P-7	P-8	P-10	P-11	P-12
P-13	P-16	P-17	P-19	P-22	P-24	P-27	P-28			

The Operator signed up for Linear Free on Sunday night. Monday morning the workspace is empty: one team named after the company, a default workflow with five states nobody chose, a sample project nobody asked for, and a blinking cursor where the first issue should go. Engineer A and Engineer B are waiting. Leadership wants to know what the team is working on by Friday. The Operator has six prior tools’ worth of dead muscle memory and no idea which Linear feature actually matters yet.

This chapter is for that Monday. It is the minimum install — a workspace a 3-person team can stand up in about a week on Free, with no upgrade and no features the team will outgrow in a week. The install is deliberately small. The overbuild test (P-24) is the filter: every label, view, and template has to earn its slot by helping the Operator decide faster or preserving evidence the team would otherwise lose.

By Friday the workspace should hold one team, nine workflow states, four labels, four shared views, one project with one update, and a GitHub PR linked to the issue it implements. That’s the install. Everything else waits — and most of it waits *on Free*, not behind a paywall.

What Free actually gives you

Linear Free is far more capable than its price suggests, and the first job of this chapter is to correct a misreading earlier drafts of the Method made: most of what the Method leans on is already on Free. Free gives you up to two teams (start with one, named for its remit); the nine-state workflow; namespaced labels; shared custom views; projects with milestones and documents; project

and initiative updates; the full objective layer — **Initiatives**, with health rollup and their own update cadence; **Cycles**; **Customer Requests** as a core intake type; **Releases** with up to fifteen pipelines; **Pulse** update digests; native issue relations (blocking, related, duplicate); sub-issues; templates; the Triage inbox; priority and estimates; GitHub and GitLab PR linking; and the GraphQL API with webhooks. The objective, intake, cadence, and release layers the Method runs on are not behind a paywall. They are on the free tier, waiting for the team to need them.

What Free actually withholds is narrower than it looks, and it falls into two buckets. The first is **scale**: Free caps you at two teams, 250 issues, and 10 MB uploads. Those are the real ceilings — mechanical limits, not missing capabilities. The second is a short list of **analytics and automation surfaces**: Insights (the queryable analytics layer on views) is Business, not Free; Triage Intelligence, Linear Asks, and SLA timers are Business; Dashboards are Enterprise. Admin roles and the lift on the caps arrive with Basic. The feature-tier matrix carries the authoritative gating — when in doubt, read it, don't guess. Net: nothing in the objective, intake, cadence, or release layer costs money. What you pay for later is room to grow (teams, issues) and the analytics and automation that sit on top of the primitives — never the primitives themselves.

The trap on Free is not scarcity; it is the opposite. Because the objective layer, Cycles, Customer Requests, and Releases are all sitting right there, the temptation is to install all of them in week one. Don't. A 3-person team that turns on initiatives, cycles, customer requests, and release pipelines on Monday has built six surfaces it cannot yet keep current. The overbuild test ([P-24](#)) applies harder on Free than anywhere, precisely because Free no longer stops you. The Method's discipline ([P-16](#), [P-24](#)) is the only thing that does. Chapter 7 covers the Basic upgrade and what triggers it; Chapter 8 covers Business and where self-install starts to break.

Week one install

Monday — team and workflow states

Rename the default team. The team's name is its actual remit — “Platform,” “Growth,” “Apps” — not the company name. The company is the workspace; the team is what this group of three is on the hook for. If the remit changes in six months, the team gets renamed. The Operator, Engineer A, and Engineer B are the only members; nobody else has a seat yet.

Then configure the workflow. Linear’s default is close to what the Method wants but not exact. The canonical nine states (P-10) are:

1. **Triage** — new, unprocessed work. Default landing zone for anything filed without a project.
2. **Backlog** — accepted, not scheduled.
3. **Ready** — scoped, available to start.
4. **In Progress** — someone is actively working.
5. **In Review** — PR or artifact awaiting review.
6. **Validation** — merged or deployed work that still needs runtime, browser, release, or human-acceptance evidence.
7. **Blocked** — someone or something else is in the way; the blocker is named in a comment.
8. **Done** — evidence on file.
9. **Canceled** — no longer matters; the reason is in a comment.

The state to install carefully is **Validation** (P-6). Install it as a manual gate. Even where the integration can move issues automatically on PR events, the Method keeps the move into Validation — and the move out of it to Done — a human act (P-7); no automation closes the loop for you. Merged code is *implemented*, not *trusted*. The gap between “the PR is in main” and “the change actually works in the environment users hit” is where the Method earns its keep. Without Validation, the team moves issues straight from In Review to Done on PR merge, and the workspace starts lying within a month. With Validation, the Operator owns the closing act: confirm the deploy, paste the evidence link, mark the issue Done. The state is the prompt.

Settings → Teams → [team name] → Workflow. Rename what you can, create what’s missing, archive what’s extra. Resist `stage:*` labels for now (P-11); on Free, the team doesn’t need them yet.

Tuesday — labels and first project

A minimal label namespace, per the chapter on labels and views (Chapter 5). Four namespaces, nothing else:

- `type:` — Feature, Bug, Chore, Documentation, Research. What kind of work this is.
- `component:` — the service or area of the codebase: `component:api`, `component:web`, `component:auth`. Fill in your own components; don’t pre-populate twenty.

- **risk:** — low, medium, high. The Operator’s prompt to surface risk early (P-13).
- **source:** — github, slack, email, support. Where the work originated. Valuable even on day one for spotting where intake noise is coming from.

Skip **domain:**, **stage:**, **parking-reason:**, **okr:**, and **release:** for now. The over-build test (P-24) asks five questions; on a 3-person team in week one, those namespaces fail at least three of them. Add labels when a view needs them — which is the test (P-16). A label not cited by any view is dead weight.

Then create the first project. One project. Name it for the team’s current focus — “Internal Product MVP,” “Platform Reliability,” “First Revenue.” Set a lead (the Operator), a start date (today), and a target date that’s honest — four to eight weeks out, not a quarter. Write a one-paragraph summary: what is being built, by when, why it matters. Link the relevant design doc, PRD, or planning note (P-5 — deep context lives outside Linear). No milestones yet; the team hasn’t shipped anything to learn what milestones it wants.

Wednesday — views and first issues

Four shared views. No more.

- **Operator’s Now** — **state** in {In Progress, In Review, Validation}, sorted by priority then updated. The Operator’s single-pane view. If a thing is moving, it’s here.
- **Waiting on the Operator** — **assignee** = Operator AND **state** in {Ready, Blocked}. The Operator’s own queue. Priority, taste, risk, and commitments don’t delegate (P-13); this view keeps the Operator’s backlog from hiding under engineering work.
- **Blocked** — **state** = Blocked, all issues, sorted by updated descending. The stuck-work radar. After three days the Operator either unblocks it or escalates.
- **In Review** — **state** = In Review, all issues. Engineering’s review queue. Catches PRs that have been open too long.

That’s it. The temptation on Wednesday is to keep going — “Stale Work,” “By Component,” “By Assignee,” “Recently Done.” Don’t. Save the impulse for two weeks from now, after the team has used these four and the Operator can point at a real decision they couldn’t make because no view supported it. That’s the bar (P-24).

Then write the first batch of issues. Pull them from wherever the team's current work lives — a planning doc, a notes file, a half-built backlog in the old tool. The rule is deliverable-sized (P-4): one outcome, defined scope, exit criteria, one owner. An issue that takes longer than two weeks to verify gets split into a parent with sub-issues (P-12). Use an issue template (P-17) — Linear's default is fine for week one. Attach the issues to the project. Engineer A picks up the first; Engineer B picks up the second.

Thursday — GitHub integration

Settings → Integrations → GitHub. Install the integration; Free supports linking multiple repos. Authorize Linear; pick which repos to connect.

The convention to install is the issue ID in the branch name or PR title. Linear auto-generates branch names from issues — Engineer A copies the suggested branch name (e.g., `eng-a/eng-42-add-rate-limiter`) when starting work. When the PR opens, Linear adds the link to the issue. That's the whole pattern on Free.

What the Method deliberately does **not** want — on any tier — is auto-transition to Done on PR merge (P-6, P-7). The Operator runs the closing by hand: PR merges → Operator moves the issue to Validation → Operator confirms deployment (browser check, deploy log link, runtime confirmation) → Operator pastes evidence in a comment → Operator moves the issue to Done. The discipline travels by hand. Chapter 10 covers the Validation gate in depth; Chapter 12 covers the deeper code-host integration. The install here is the minimum loop that produces an evidence chain (P-8, P-28).

GitLab equivalent. Enable the GitLab integration, use the issue ID in branch names and MR titles, and run the Validation close manually. The mechanic differs by integration; the doctrine doesn't.

AUGMENTATION SURFACE · Install Verification

Where an agent layer makes this work better: at the end of the install week, an agent can scan the workspace against the install checklist — confirming the nine workflow states exist with the right categories, the four views are shared at the team scope, the GitHub integration is wired with at least one linked PR, the first project has a target date and at least one issue, and the first project update is posted. The agent surfaces gaps as a list, not a fix.

What the PM keeps: the judgment call on whether a deferred item — a label namespace, a fifth view, Cycles, a first initiative — should be added in week two or left until the team has a decision that needs it. The agent enumerates; the Operator decides.

Friday — first project update

Engineer A's PR opens Friday afternoon. The Operator catches it in the In Review view, reviews the PR, and waits for merge. Once merged, the Operator moves the issue to Validation, pastes the deploy log link, runs a browser check on the staging or production URL, pastes the screenshot, and moves the issue to Done. The full evidence chain (P-28) shows up on the issue for the first time. The Operator now knows what the loop costs in minutes and has a real example to show Engineer B on Monday.

Then the Friday project update (P-19). Linear's update format is enough on Free. Five short sections:

- **Shipped.** What got to Done this week, with the evidence link.
- **Open.** What's in flight, by name.
- **Blocked.** What's stuck and why.
- **Decisions needed.** What the Operator needs from leadership (or from inside the team) to keep moving.
- **Next.** What gets started next week.

Post it. That update is the artifact leadership consumes (P-27) and the cadence the Method runs on for the lifetime of the project — every Friday, no exceptions, even when the update reads “shipped: nothing; the team spent the week stuck on X.”

Growing into the rest — still on Free

The week-one install is four primitives plus one project and one integration. The next several weeks are about reaching — one capability at a time, each when a real decision needs it (P-16) — for surfaces Free already includes, no upgrade required:

- **Initiatives** when Leadership starts asking how projects ladder up to outcomes (P-2). Two or three durable initiatives, owners, quarter-end targets. The objective layer is Free; reach for it the week the rollup question lands, not before.
- **Cycles** when the team wants a time-boxed cadence to commit against rather than an open backlog. Whether a 3-person team needs cycles, or runs on the weekly update cadence alone, is a judgment call — and it is free either way.
- **Customer Requests** when feedback arrives faster than the Operator can hand-convert it and provenance starts getting lost (P-22). A request is captured in-take, never automatic scope.
- **Releases** when the team ships on a pipeline worth naming. Free covers up to fifteen pipelines — more than a small team needs; the release-organization doctrine lives in the code-host chapter (Chapter 12).
- **Pulse** when the Operator wants update digests pushed rather than pulled.
- **Native relations** the first time “Blocked” as a state is too coarse — when an issue is blocked *by* a specific other issue and the link itself is the information.

None of these is an upgrade. Each is a Free primitive the team grows into when a view or a decision pulls it in — and each still answers to the overbuild test.

What you have at the end of week one

A workspace Engineer B can open on Monday and understand. One team named for the team’s remit. Nine workflow states, with Validation installed and used at least once. Four labels, each cited by at least one view or attached to at least one issue. Four shared views covering the Operator’s morning, the Operator’s own queue, the team’s blocked work, and the team’s review queue. One project with one update and at least one Done issue carrying full evidence. A GitHub integration with at least one linked PR. A weekly rhythm the team has run end-to-end once.

The rhythm to keep, lightly (Chapter 13 covers reporting in depth): Monday the Operator confirms priorities and clears Triage. Daily the Operator clears Waiting on the Operator and Blocked. Friday the Operator posts the project update.

Ceiling signal

Tier 1's ceiling is mechanical, not feature-shaped — and that is the whole correction. The team does not leave Free because it needs Initiatives, Cycles, Customer Requests, or Releases; it already has them. It leaves Free when it hits the caps. Three concrete signals; when any one of them is firm, move to Chapter 7 and the Basic upgrade:

- **The issue count is approaching 250.** Free's hard cap. The Operator notices archived-versus-active math starting to matter, or Linear warns that new issues are near the limit. Basic lifts this to unlimited.
- **The team needs a third team.** Free allows two. A second squad with its own remit, or a separate team for a distinct product surface, is the moment the 2-team cap binds. Basic raises it to five.
- **Administration outgrows flat membership.** The workspace needs admin roles to delegate settings, or uploads are bumping the 10 MB ceiling. Both come with Basic.

Unlike the later tier moves, there is no “two of three” threshold here. These are hard limits, not judgment calls — you cross a line or you don't. One firm cap signal is enough.

WHERE THIS GETS HARD

The Validation gate assumes the Operator can produce the evidence — a deploy log link, a browser check, a runtime confirmation. When the team needs Validation evidence from infrastructure the Operator doesn't own — a separate ops team owns the deploy pipeline, the deploy logs live in a system the Operator can't read, getting a link on every production issue requires negotiating with another team's PM — the install stalls. The workspace looks correct; the evidence chain (P-28) breaks at the boundary.

Ravenhelm assesses your operating surface and installs the Method tuned to your team, your code host, and your reporting structure. nate@ravenhelm.co

Tier 2: Linear Basic Upgrade

PRINCIPLES CITED

P-2

P-18

P-22

P-24

It is month four. The Tier 1 install from Chapter 6 has held. The Operator has a handful of active projects, two initiatives wired above them, a clean Triage queue most mornings, weekly updates nobody has had to chase, and a GitHub link on roughly every issue that should have one. The system is real, and — this is the part the earlier draft of the Method got wrong — almost all of it was built on Free.

Then two things happen in the same week. The Operator goes to file a batch of new issues and Linear warns that the workspace is closing on 250 — Free's hard cap. And Leadership greenlights a second squad: a small team to own the public surface, with its own remit and its own lead. The Operator goes to create the team and finds Free allows two, and both slots are spoken for the moment the new team exists.

Neither problem is a missing feature. Both are walls. That is what Tier 2 is for.

What Basic actually adds

Be blunt about this, because the previous tiering oversold it. Linear Basic, at \$10 per user per month, adds exactly four things over Free, and all four are ceilings lifted, not capabilities unlocked:

- **Five teams**, up from two.
- **Unlimited issues**, up from the 250-issue cap.
- **Unlimited uploads**, up from 10 MB.
- **Admin roles** — scoped administration so someone other than the Operator can manage a team without owning the whole workspace.

That is the entire list. Basic is a ceiling lift. It is the tier you buy when the team has outgrown Free's *limits*, not when it wants new *machinery*. If you are reaching for Basic expecting it to switch on a layer of the Method, you have misread the price sheet — and the next two sections are the correction.

The misconception to drop

The earlier draft of this document attributed Initiatives, Customer Requests, and the trend layer to “Plus.” That was wrong, and the correction matters because it changes what you buy and why.

Initiatives are Free. The objective layer (P-2) — the durable container above projects, with health and its own update cadence — is on the free tier. The Operator already installed it back on Free. You do not upgrade for it.

Customer Requests are Free. Structured voice-of-customer intake with attribution and impact (P-22) is on the free tier too. You do not upgrade for it.

Cycles, Releases, and Pulse are Free. The cadence container, release pipelines (up to fifteen), and update digests are all free. You do not upgrade for any of them.

Insights is *not* on Basic. The queryable analytics layer on views — the thing that replaces the Operator's Friday spreadsheet — is a **Business** capability, not a Basic one (Chapter 8). So are Triage Intelligence, Linear Asks, and SLAs. Dashboards are Enterprise (Chapter 9). The feature-tier matrix carries the authoritative gating; read it before you upgrade for a feature, because Basic almost certainly does not contain the feature you have in mind.

Net: there is no layer of the Method that switches on at Basic. The objective, intake, cadence, and release layers are already running on Free; the analytics and automation layers are still a tier away on Business. Basic sits in between, and what it sells is room.

The upgrade is administrative, not architectural

Because Basic adds no machinery, the upgrade changes almost nothing about the workspace the Operator has been running. The nine states, the labels, the shared views, the initiatives, the customer requests, the release pipelines — all of it keeps working exactly as it did the day before, unchanged. Treat the upgrade as a quiet infrastructure event, not a re-platforming. Nothing needs re-stalling.

What does change is operational headroom. Issues stop bumping the cap, so the Operator can stop archiving aggressively just to stay under 250. The third and fourth teams become creatable, so the second squad gets a real home instead of a shared-team workaround. And admin roles become available — the one addition worth a paragraph of its own.

Admin roles — the one genuine new lever

Admin roles are the only thing on Basic that is a capability rather than a relaxed limit, and even this is modest. On Free with one team and three people, a flat membership is fine: everyone the Operator trusts can touch settings, and there is little to break. The moment the workspace has a second team with its own lead, that stops scaling. The new team's lead needs to manage *their* team's workflow, labels, and members without being able to reconfigure — or delete — the rest of the workspace.

Admin roles let the Operator delegate scoped administration: a team admin who owns one team's settings, an org-level admin who can manage members and billing. Grant them the same way you add anything else under the Method — minimally, and only where someone is actually administering (P-24). An admin role handed out “just in case” is overbuild with a blast radius. The default stays: the Operator owns the workspace; admin roles are how the Operator stops being the only person who *can* own it.

Standing up the second team

The clearest thing Basic actually buys is room to run the second squad as a real team instead of a workaround. On Free, a team that hit the 2-team cap had to cram a new squad into an existing team and fake the separation with a label or a project — never a real boundary. Basic removes that compromise. Standing up team three (or four, or five) is mechanical, and the Method's move is to clone the operating surface rather than reinvent it:

- **Copy the spine.** The new team gets the same nine states and the same namespaced labels, so a member who knows one team can read the other on sight.
- **Seat the lead as a team admin.** The new squad's lead owns their team's settings without touching the rest of the workspace — the one place admin roles earn their keep.

- **Wire under the initiatives that already exist (P-2).** Two squads under one initiative is the normal shape; don't spin up parallel objectives just because there is now a second team.

Locality still applies (P-18): the lead can tune a label or a view that only makes sense for their remit, but the spine stays shared. A workspace where every team invents its own surface is exactly what Chapter 5 was written to prevent.

AUGMENTATION SURFACE • Seat and Team Watch

Where an agent layer makes this work better: watching the team count against Basic's five-team ceiling, flagging dormant members who hold seats the workspace is paying for, and surfacing the access map — who has an admin role on which team, and whether anyone is over-scoped. On Basic the meter runs per seat, so seat hygiene is cost hygiene.

What the PM keeps: the call on who gets a seat, who gets an admin role, and when a fifth team means it is time to look at Business rather than provision blindly toward the ceiling.

Ceiling signal — how to know Basic is no longer enough

Basic's headroom is wide on issues and narrow on everything else, so it runs out in one of two ways — and the difference is the whole point. Either the team needs more room than Basic gives, or, far more often, the team needs *capability* Basic was never going to provide. When two of the following land in the same quarter, plan the move to Chapter 8 and Business:

- **Intake or reporting that needs intelligence, not headroom.** The Operator wants Insights to stop rebuilding the same Friday chart, or Triage Intelligence to route a Triage queue that has outgrown a thirty-minute daily pass, or Linear Asks to capture internal channels the Operator no longer monitors in real time. None of those is on Basic; all of them are on Business.
- **Work that needs a clock.** Production traffic, urgent regressions, decisions with deadlines — work that needs an SLA, which Basic does not have.
- **The team passes five teams, or needs a boundary.** A sixth team, a private team for sensitive work, an outside guest — the team has outgrown Basic's team model and wants the boundaries and access controls that start at Business.

The honest framing: the move from Free to Basic was about ceilings. The move from Basic to Business is the first time since Free that you pay for *capability* —

and that makes it the more consequential upgrade. Basic buys you time; Business buys you machinery.

WHERE THIS GETS HARD

The team that upgrades to Basic expecting it to “unlock the Method” — switch on the reporting, the intelligence, the automation — discovers Basic does none of that, and can feel like it paid \$10 a head for nothing. It didn’t: it bought the room it actually needed. But if what the team needed was Insights or Triage Intelligence or SLAs, Basic was the wrong purchase and Business was the target all along. Knowing which wall you are actually hitting — a limit or a capability gap — is the difference between a \$10 upgrade that buys headroom and a \$16 upgrade that buys the machine.

Ravenhelm assesses your operating surface and installs the Method tuned to your team, your code host, and your reporting structure. nate@ravenhelm.co

Tier 3: Linear Business and the Edge of Self-Install

PRINCIPLES CITED P-6 P-7 P-8 P-9 P-13 P-14 P-15 P-17 P-18 P-20 P-22
P-24 P-25 P-27

It is the end of the Basic quarter. The Operator opens Linear on a Tuesday morning and counts twenty items in Triage that came in overnight — a Sentry alert that fired at 3 a.m., four bug reports from beta users in Customer Requests, a dozen Slack-forwarded asks that an engineer has been pasting into Triage manually since Friday, and three “quick question” emails an executive forwarded with no context. There is a production incident from yesterday that should have had a clock on it and didn’t. Engineer A is heads-down on a research repo and has been ignoring Triage for a week because the Operator told them to. Engineer B owns the public surface that just shipped and is waiting for the Operator to tell them which of the four beta-user bugs is real.

The Operator does the Triage pass. It takes ninety minutes. By the time it is done, the morning is gone, the Friday update for Leadership is unwritten, and a request the Operator already promised a decision on is now two days late.

This is the moment Basic stops being enough.

The shift Business represents

Tier 1 was self-install on Free — and Free already carried the objective, intake, cadence, and release layers the Method runs on. Tier 2 (Basic) lifted Free’s ceilings: more teams, unlimited issues, admin roles. Neither tier added a genuinely new capability surface. Business is different. Business is the first tier since Free that you buy for *capability* rather than headroom, and it is where Linear hands the Operator machinery the Operator does not have time to tune alone. Insights

have to be shaped against the questions Leadership actually asks. Triage Intelligence has to be trained against the team's real patterns. Asks templates have to be designed so the form does not become a tax. SLAs have to be calibrated against actual incident history, not a generic table copied from a blog post. Agent surfaces — Linear Agent, AI Agents, MCP, Code Intelligence — appear as primitives, and someone has to decide what the agent through them actually does.

The capabilities are real. The ceiling here is not Linear; it is the Operator's bandwidth. That is where the freemium document stops giving away the full method. The shape is here. The tuning that makes the shape trustworthy is engagement-shaped work.

Insights — the first analytics layer

Insights is the first genuinely new analytics surface since Free, and naming it here is a load-bearing correction: the Method's earlier tiering placed Insights a tier too low, at "Plus." It is a **Business** capability. Free gives you views — a view is a saved filter, a question about the current state of the workspace. Basic adds no analytics at all. Business adds Insights: that same filter *over time*, or sliced by a property, with a measure attached. Issue count by status over the last 30 days. Cycle time by project. Throughput per engineer. Triage time by source. Pin an Insight to a view and it updates itself.

The Operator's first install is two Insights, both pinned to the Operator's Now view:

1. **Open issue count by status, last 30 days.** The shape of work in flight — whether Triage is climbing, whether Validation is backing up, whether In Review has become a parking lot.
2. **Cycle time by project, last 60 days.** Whether the team is getting faster or slower at the same shape of work, project by project.

These replace what the Operator did by hand for the Friday summary all the way back on Free. Leadership reads the summary; they do not browse the workspace (P-27). An Insight lands in the Friday note only when it is moving in a direction that needs explaining. Avoid reporting theater: an Insight that never changes the Operator's plan is a candidate for deletion (P-24).

Composed, multi-view **Dashboards** — the pinned board Leadership opens for itself — are an **Enterprise** capability, not a Business one; the Enterprise chapter (Chapter 9) covers them. On Business, the unit of reporting is the individual In-

sight panel and the Friday summary the Operator writes around it. What the chapter does not cover: matching Leadership's reporting taste — which slices, which groupings, which properties the workspace does not yet capture. Iterating an Insight until it earns weekly attention is engagement-shaped work.

Triage Intelligence

Triage Intelligence is Linear's AI-assisted routing layer. When a new item lands in Triage — from Slack, email, GitHub, Sentry, a customer request, a manual entry — Triage Intelligence proposes labels, a project, an assignee, related issues, and possible duplicates. The Operator confirms or overrides; over time the system gets better at the team's particular shape of work. Alongside it, Business adds **Triage Rules** and **Responsibility** — deterministic routing and an on-call rotation for who owns the inbox when — so the queue is not implicitly always the Operator's.

The Method's posture is that Triage Intelligence starts conservative. Suggestions, not auto-application. Labels and duplicate flags can auto-apply early because the cost of being wrong is low and the cost of policing is low (P-25). Project, assignee, priority, and cancellation stay suggestion-only until the Operator has watched the system make those calls correctly for two or three weeks. Customer feedback never auto-converts to scope — the principle is older than the feature (P-22). An item from Customer Requests can be enriched with suggestions, but conversion to an issue or project remains an Operator decision.

When it pays for itself: the day the Operator's daily Triage pass takes longer than thirty minutes. For the 3-person team in this chapter's opening, that day was last Tuesday.

What the chapter does not cover: the tuning. Telling Triage Intelligence which of two `component:*` labels applies to a given alert pattern, or which project owns Sentry exceptions from a particular service, or which Slack channel's asks should default to Engineer B — those decisions live in workspace and team guidance documents that have to be written, tested, and revised as the team's project shape changes. The system gets smart in proportion to the training data the Operator gives it.

AUGMENTATION SURFACE • Triage at Scale

Where an agent layer makes this work better: pre-classifying Triage items against the team's established patterns, flagging the three items the Operator should look at first, and escalating the ones that need human judgment within the seven-day Triage SLA (P-20). On a twenty-item morning, the Operator opens a sorted queue with summaries already drafted instead of twenty cold cards.

What the PM keeps: every escalation, every priority change, every rejection. The agent layer proposes; the Operator decides. Delegation without acceptance criteria produces wasted capacity (P-15); the agent layer's job is to compose the criteria, not to skip past them.

Linear Asks

Linear Asks is structured intake from Slack and email. Where Customer Requests handles external feedback for a public-facing product surface, Asks handles internal requests: an engineer in another part of the organization asking the team to look at something, an executive forwarding a question with a deadline attached, a partner team filing a request through a Slack channel the Operator does not monitor in real time. Asks lands in Triage with a structured form already filled out — requester, channel, summary, optional template fields — so the Operator is not staring at “FYI” with no surrounding context.

For the 3-person team, the install moment is when the Operator has three Slack channels of unprocessed messages and Engineer A has stopped pasting them into Triage because pasting is now a job. Asks turns the paste into a form. The Slack channel becomes an Ask channel; form fields capture what the Operator would otherwise have to chase down in DMs. The captured ask routes through Triage and gets the same Triage Intelligence treatment as any other item.

Asks **web forms** — public-facing intake forms hosted for requesters outside the workspace — are an **Enterprise** capability, not a Business one (Chapter 9). On Business, Asks runs through Slack and email channels; that covers internal intake, which is what the 3-person team needs first.

When it pays for itself: the moment the Operator can no longer say with confidence what is sitting in the team's Slack DMs.

What the chapter does not cover: the form design. Asks templates are product design. Too few fields and the Operator still has to chase context; too many and the requester abandons the form and goes back to DM. Designing a four-field

template that captures enough to triage without becoming a ten-field tax takes iteration against real requesters. Different intake sources need different templates — a bug report template is not an executive-decision template. The Method names the principle (templates lower the cost of doing the right thing, P-17); the engagement does the tuning.

SLAs

SLAs are turnaround clocks on issues that need urgency tracking distinct from priority. Priority answers “which of these matters most”; an SLA answers “by when does this need a response.” The two are not the same. A low-priority bug with a 24-hour SLA matters less than a high-priority feature but has a clock on it; the high-priority feature does not.

The Method’s posture is that SLAs exist for a narrow set of work and the Operator owns the commitments behind them (P-13). The starting rule set is small and conservative:

- Production incident: 24 hours to first response, resolution clock tracked separately.
- Security or access blocker: 48 hours.
- Operator decision blocking active work: 2 business days.
- Customer-facing regression: 1 week.

For the 3-person team, SLAs land on the Operator’s calendar the week Engineer B’s public surface starts taking production traffic. The two engineers can no longer absorb “we’ll get to it” as an answer for that surface; customers can’t either. SLAs convert “we’ll get to it” into a clock the workspace tracks and the Friday update reports against.

When it pays for itself: the first production incident that should have had a clock and didn’t. The Operator finds out after the fact that nobody picked it up for thirty hours.

What the chapter does not cover: distinguishing real-risk SLAs from noise SLAs. Putting an SLA on every priority-high issue produces a workspace where every issue is breaching SLA and no issue is actually urgent. The calibration — which incident categories actually need a 24-hour clock, which “urgent” decisions are really 2-business-day decisions — comes from the team’s incident history. A team without six months of clean incident data is calibrating against folklore.

Verify what you enforce (P-25); SLAs that fire on the wrong things stop being trusted, and untrusted SLAs are worse than no SLAs.

Agent surfaces — agents as workspace participants

Business unlocks the agent surfaces, and this is the part of Linear that has moved fastest since the Method's first edition. The Method's job is unchanged: it describes what each surface **is** and names where an agent helps. It does not, in this document, describe how to build the agent that uses them. That distinction is the entire boundary between freemium content and engagement — and it has gotten sharper, not softer, as the surfaces have grown teeth.

AI Agents are app users. An agent registers in the workspace as a member: it has a name, it can be @-mentioned in comments, documents, project descriptions, and updates, it can be assigned or set as a delegate on an issue, and it can comment and collaborate the way a human teammate does. Linear ships its own first-party agent — addressable by @-mention for workspace questions, thread summaries, and issue drafting — and third-party agents (a coding agent on a repo, a research agent against a corpus, a QA agent against a deploy) register through the same framework rather than running as scripts fired from a terminal. The **agent framework** — registering agents, @-mentioning them, the `delegate` field — is available broadly at the framework's free tier; the **agent automations** that let those agents take workflow actions on their own are a **Business/Enterprise** capability.

The `delegate` field is the load-bearing primitive. An issue's assignee is a human; its delegate may be an agent. That split is not cosmetic — it is P-14 rendered as a field. The assignee is the throat to choke; the delegate is who is doing the typing this week. Setting the assignee to a bot makes the issue ownerless; setting the delegate to a bot keeps a human accountable while the agent works (P-14). Delegation to an agent still requires the full scope packet — outcome, scope, validation command, expected artifact, repo and file paths — that delegation to a human requires, and arguably more, because the agent has no judgment to fall back on (P-15).

Agent guidance and shared Skills are how the Operator standardizes agent behavior without re-typing instructions into every issue. Workspace- and team-level guidance documents hold the standing instructions an agent reads before it acts; **Skills** package the things the Operator does repeatedly — the shape of a good triage summary, the format of a delegation packet, the house rules for a release note — into reusable units an agent can invoke. Guidance is where locality

lives for agents: a repo-strict convention or a team-specific default belongs in the closest guidance layer, not the global one (P-18).

MCP server support is the protocol surface that lets Linear’s agents and external agents reach tools and context outside Linear — the code host, the documentation system, the monitoring stack — through approved MCP servers configured at the workspace level. MCP is what turns an agent from a chat box that knows only the issue text into a participant that can read the repo, the runbook, and the last deploy log before it answers.

Write-with-Agent drafts the prose the Operator would otherwise write by hand — a project or initiative update composed from the issues and evidence already in the workspace. The agent drafts; the Operator owns the send. A project update is a commitment Leadership reads as truth (P-27), and priority, taste, and risk do not delegate (P-13): the agent can assemble the words, but the Operator decides what the update claims.

Triage Automations let an agent take open-ended actions on Triage items — enrich, label, route, flag a duplicate, draft a first response — inside the seven-day Triage SLA (P-20). The posture matches Triage Intelligence above: labels and duplicate flags can auto-apply early because the cost of being wrong is low; conversion, priority, and cancellation stay human until the Operator has watched the automation make those calls correctly for weeks (P-22, P-25).

The escalation — agents that write and ship code

The Method’s first edition treated agents as planning and coordination help: they summarized threads, drafted issues, pre-classified Triage. That framing is now out of date. Linear’s agents have escalated from drafting comments to **writing and shipping code**, and the discipline this chapter describes does not loosen as a result — it tightens. An agent that drafts a comment costs a bad paragraph when it is wrong. An agent that opens a pull request costs a bad merge. The higher the blast radius, the more load-bearing P-14 becomes.

Code Intelligence (Business/Enterprise, beta) is the first rung: an agent that reads the connected codebase and answers implementation questions grounded in the actual repo rather than the issue text alone — “where is rate limiting enforced,” “what breaks if we change this schema.” It reads; it does not write. Its value to the Method is that it makes delegation packets sharper: the acceptance criteria and validation command (P-15) can be written against what the code actually does.

Coding Sessions (Business/Enterprise) are the escalation itself. An agent — running Claude Code or Codex, cloud-executed rather than tied to an engineer’s laptop — takes an issue, writes the code, and opens a pull request: triage-to-fix with no human keystroke between the report and the diff. This is the capability that changes the stakes of everything above it. The agent is no longer proposing work; it is producing the artifact P-6 says must be validated. A Coding Session that opens a PR has produced *implemented*, not *trusted*, code (P-6); the code host still proves the implementation (P-8); and nothing customer-facing reaches Done without release evidence, however the diff was authored (P-9).

Linear Diffs bring code review inside Linear — the diff, the review threads, and the merge decision surfaced next to the issue rather than only in the code host. When the author of the diff is an agent, the review is not a courtesy; it is the validation gate (P-6), and the human reviewer supplies the acceptance the evidence chain requires from someone other than the author (P-7).

“Open in coding tool” is the low-ceremony version of the same move: from any issue, the Operator or an engineer hands the work to Cursor, Claude Code, or Codex with a prompt template pre-filled from the issue. It is a **Free-tier** convenience — no Business seat required — and it is the most common on-ramp, because the human stays in the driver’s seat the whole way: the agent writes in the tool, the human runs the validation command and opens the PR under their own name.

For the 3-person team, the escalation lands the week Engineer A stops pasting an issue into a coding tool by hand and instead delegates the sub-issue to a Coding Session on the research repo. The issue stays assigned to Engineer A (P-14); the delegate field carries the agent; the acceptance criteria and the validation command are in the issue body (P-15). The Coding Session opens a PR, a Linear Diff surfaces it, and Engineer A reviews, runs the validation command, and moves the issue to Validation with evidence (P-6, P-7). The agent wrote the code. Engineer A still owns whether it ships — which is exactly why the escalation makes the human-ownership discipline matter more, not less.

When it pays for itself: never, automatically. The surfaces pay only when there is a delegation protocol around them. Buying Business for the agent surfaces and skipping the protocol is buying a self-driving car with nobody willing to sit in the seat — and now the car merges to main.

What the chapter does not cover: how to build the agent. Prompt design, skill chains, MCP-server selection, orchestration patterns, the AI PM that composes delegation packets from Triage Intelligence’s suggestions and audits a Coding

Session's output before a human ever opens it — those are Ravenhelm product, not freemium content. Linear ships the surfaces; the Method names where they help; the agent that runs safely through them is engagement-shaped work.

Team boundaries, guests, and access controls

Business is also where the workspace gains boundaries. **Private teams** let a team's issues stay invisible to the rest of the workspace — the first time a sensitive workstream (security, an acquisition, a comp project) can live in Linear without a separate workspace. **Guests** let an outside collaborator — a contractor, a partner-team reviewer — into specific teams without a full seat across the workspace. **Login-method restrictions**, including enforced Google SSO, let the Operator require a sanctioned sign-in path. And the **Intercom** and **Zendesk** integrations route support conversations into Customer Requests, so the feedback layer the team has run on Free since the beta now connects to the channels support actually lives in.

What stays above Business, in Enterprise (Chapter 9): SAML/SSO with SCIM provisioning, the audit log, IP-range restrictions, multi-level sub-teams, Dashboards, Asks web forms, and the Salesforce/Gong/Airbyte integrations. The line between Business and Enterprise is the line between one well-run organization and a fleet of teams that has to be governed centrally.

Ceiling signal — when the Operator's bandwidth runs out

Tier 3's ceiling is not Linear features. The features are there. The ceiling is the Operator. The signals to watch for are bandwidth signals, not capability gaps:

- **Tuning takes more time than the Operator has.** Triage Intelligence is drifting because nobody has written the workspace guidance update it needs. Asks templates have not been touched in two months and intake quality is degrading. SLA rules are firing on things that shouldn't have SLAs because the calibration pass keeps slipping. The workspace oscillates between hands-on weeks when the Operator catches up and neglected weeks when other priorities consume the calendar. Nothing is broken; everything is decaying.
- **Leadership asks for reporting the Operator can produce once but not sustain weekly.** The first quarterly business review goes well — the Operator spent a Sunday building it. The second one is late. The third one is missing data. The Insight panels Leadership wanted to “just open” have stale numbers because the underlying view configuration drifted and the Operator has not had the

afternoon to fix it (P-27). When Leadership wants a board it can pull up unaided, the answer is an Enterprise Dashboard (Chapter 9) — and someone has to own keeping it current.

- **The team starts hiring around the gap.** A second Operator, a Chief of Staff, an embedded TPM, an agent layer the Operator does not have time to design — the team is solving for the ceiling by adding headcount or capability without first installing the operating model the new role plugs into. The new hire spends their first quarter inventing the same set of views, the same Triage routine, the same delegation protocol the existing Operator was running in their head.

When any one of those signals lands, Ravenhelm starts. The Method's freemium document gives the shape; the engagement gives the fit. A 3-person team can install Tier 1 alone on Free in a week and grow into Basic in a quarter. Tier 3 is where alone runs out. The tier beyond it — Enterprise (Chapter 9) — is not about more capability for one team; it is about running the method across many teams at once.

WHERE THIS GETS HARD

The team that wants Business-tier capabilities operational on day one — Triage Intelligence routing real intake, Asks templates capturing real channels, SLAs firing on real incidents, Insights Leadership actually reads, agents participating in real delegation loops — has no calendar for the multi-week tuning that makes any of it trustworthy. The capabilities ship in an afternoon; the trust takes a season.

Ravenhelm assesses your operating surface and installs the Method tuned to your team, your code host, and your reporting structure. nate@ravenhelm.co

Tier 4: Linear Enterprise and the Method as Operating System

PRINCIPLES CITED P-6 P-10 P-14 P-15 P-16 P-17 P-18 P-19 P-22 P-23 P-24
P-25 P-27 P-28

It is the Monday after the second acquisition closed. The Operator who installed Free in Chapter 6 and tuned Business in Chapter 8 now sits a rung up the ladder (Chapter 4) — they run the operating model, not a single team. The workspace they once held in their head has become forty-one teams across six departments and two newly absorbed companies. Three things land before the first coffee. Security forwards the SOC 2 readiness checklist: the auditor wants SAML enforced, a provisioning trail, and proof that the contractor who rolled off in March no longer has a seat — he still does. Leadership asks, for the third week running, for “one view across all of engineering,” and the Operator cannot build it, because each team’s workspace has drifted: the acquired team brought fourteen workflow states from Jira, one product group quietly redefined Done as merged, and “Validation” means three different things in three different places. A staff engineer pastes a compensation-planning thread into a public project channel by accident.

None of these is a feature gap. Every one is a scale gap. The Method that one Operator ran for three people now has to run as infrastructure for hundreds — without fragmenting into forty-one private dialects of itself.

This is the moment Business stops being enough.

The shift Enterprise represents

Everything in this document up to here assumed one Operator and a small number of teams. The install was something a single person could hold: nine states

they chose, four views they read every morning, one evidence chain they walked by hand. Enterprise breaks that assumption. The problem this tier solves is not “what feature does the team lack” — it is “how does one operator’s method become an organization’s operating system without splintering into incompatible local versions.” The lower tiers — Free (Chapter 6), Basic (Chapter 7), Business (Chapter 8) — are about installing the Method for a team. Enterprise is about governing it across people who did not install it, did not read this document, and arrived with muscle memory from a different tool.

Linear’s Enterprise tier is the substrate for that governance: identity and audit controls that satisfy a security review, a team topology (sub-teams, private teams, guests) that can model an actual org, workspace-level Dashboards that see across every team at once, and shared agent guidance that lets a central team teach the rest. The features are the easy half. The operating shape that uses them is the hard half — and it is the half a single Operator cannot stand up alone, which is the entire reason this is the tier where the Method becomes an engagement rather than a self-install.

The federated operating model — hub, spokes, and a community of practice

The shape that scales the Method across many teams is **federated, not centralized and not anarchic**. A central team — the hub — owns the standards: the nine-state canon (P-10), the shared label namespace (P-16), the issue and update templates (P-17), the Validation discipline (P-6), the principle set itself. The hub does not do the delivery work; it owns the guardrails the work runs inside. The execution teams — the spokes — run their own backlogs, projects, and priorities within those guardrails. And a cross-team **community of practice** — the Operators of every spoke, meeting on a cadence — is where templates, views, and hard-won lessons move between teams instead of being reinvented in each one.

This maps cleanly onto **Locality wins** (P-18), the load-bearing principle of the whole tier. The hub sets workspace-level baselines; a spoke overrides them where its work genuinely differs — the security team’s Done is stricter, the research spoke’s cycle is longer — and the override is legible precisely because the baseline it departs from is written down. Federation without P-18 is either a straitjacket (every team forced into one shape) or a free-for-all (every team its own dialect). With P-18, the hub publishes the default and the spoke earns its exceptions. The community of practice is where an exception that proves itself in

one spoke gets promoted back into the hub's default for everyone — the Method improving itself instead of ossifying.

AUGMENTATION SURFACE · Federation Conformance

Where an agent layer makes this work better: scanning every team's workspace against the hub's published baseline — flagging the team that added a tenth workflow state (P-10), the spoke whose Done has no Validation gate (P-6), the labels no view cites (P-16), the projects three weeks stale (P-19) — and posting the drift as a conformance report to the community of practice, not a fix.

What the PM keeps: deciding which drift is a violation to correct and which is a justified local override (P-18) worth promoting into the hub's baseline. The agent measures conformance; the humans decide what conformity is worth.

Identity and the audit trail — SSO, SCIM, restrictions, audit log

Enterprise is where Linear answers a security review. **SAML SSO** routes login through the organization's identity provider, so a person's Linear access *is* their corporate identity — revoke the identity, revoke the workspace. **SCIM provisioning** makes that automatic: deprovisioning in the identity provider deprovisions the seat, which is the control that would have closed the contractor's account in March without anyone remembering to. **IP restrictions** and **login-method restrictions** (the latter also available on Business) narrow how and from where the workspace can be reached. The **audit log** records who changed what and when — the provisioning trail the auditor asked for. The **Workspace Owner** role and **third-party app review** put a named human in the approval path for every OAuth app, MCP server, and agent that wants into the workspace (P-14).

The Method's posture is that these controls are how *Verify what you enforce* (P-25) scales past one person. At three people, the Operator *was* the access policy. At the federation's size, the policy has to be machinery: SCIM is the enforcement point for “no orphaned seats,” the audit log for “we can reconstruct what happened,” app review for “no agent enters the workspace without an owner accountable for it.”

When it pays for itself: the first security questionnaire, SOC 2 audit, or enterprise customer that makes SSO and an access trail a condition of the deal rather than a nice-to-have.

What the tier does not install is the policy itself — which identity groups map to which teams, which apps clear review, what the IP allowlist actually is. That mapping is security-and-operations design done against your real identity provider and your real org chart.

Sub-teams, private teams, and guests — the federation’s structure

Three team primitives let the workspace model the actual organization. **Multi-level sub-teams** (nested up to five levels) are the Enterprise addition that makes the hub-and-spoke topology real: a parent team for a department, sub-teams for each squad, with shared settings and rollups flowing up the tree. **Private teams** and **guests** arrive at Business but become load-bearing here — private teams wall off work that genuinely cannot be open (compensation, security embargoes, M&A), so the staff engineer in the opening cannot paste the comp thread where it does not belong because the channel does not exist for outsiders; guests scope external collaborators into exactly the teams they need and nothing else. Enterprise adds **private-team issue sharing**, the controlled seam that lets a walled team hand a single issue across the boundary without opening the wall.

The Method’s posture is that structure follows the operating model, not the reporting chart (P-24). Sub-teams should mirror how work and accountability actually flow, not how the org publishes its hierarchy; a private team is justified by a real confidentiality boundary, not by a manager who wants a walled garden; a guest seat is justified by real scoped collaboration, not as a cheap way to dodge a license.

When it pays for itself: the first time a single flat list of teams stops describing how the org actually works — when a department needs its own rollup, or a sensitive workstream needs a wall the rest of the workspace cannot see over.

Modeling the federation badly — sub-teams five levels deep nobody navigates, private teams hiding ordinary work from the people who need it — is the over-build test (P-24) failing at organizational scale, where it is far more expensive to unwind than it ever was for one team.

Dashboards and data out — the cross-team instrument panel

Dashboards are workspace-level pages composing panels across many teams at once — the cross-team view Leadership asked for and could not get (P-27). Where Business-tier Insights answered questions about one team’s work, a Dashboard

answers them across the whole federation: open issues by team, cycle time by department, SLA health across every spoke, Validation throughput org-wide, all self-updating. This is the hub's instrument panel — the single surface that tells the operating model whether the spokes are healthy. The reporting chapter covers the discipline of *what* belongs on it and how it rolls up; this tier is what unlocks the cross-team canvas.

Airbyte and data-warehouse sync are the other direction: Linear's data flowing out, one way, into the organization's warehouse and BI stack, where it joins finance, product analytics, and revenue data the Method never touches. The Method's posture is that this is export, not sync — consistent with *the system of record is Linear* (P-23). Webhooks and Airbyte carry data *out* to the warehouse; nothing two-way writes priority or status back in. The evidence chain (P-28) the warehouse reports on is only as honest as the issue-level evidence beneath it, which is why the pipeline is worth nothing until the spokes are actually running Validation.

When it pays for itself: the first time Leadership needs Linear's delivery data joined to numbers that live somewhere else — cost, revenue, headcount — to answer a question no single Dashboard can.

What the tier does not cover is the data model — which Linear fields map to which warehouse tables, what the BI layer is allowed to assert — engagement-shaped work against your specific stack.

Enterprise intake — Asks web forms, Salesforce, Gong

Two intake surfaces unlock here. **Asks web forms** (gated behind SAML) extend Business-tier Asks (Chapter 8) from Slack and email to public, structured web forms — an intake front door anyone in the organization, or an external requester, can file through without a Linear seat. **Salesforce and Gong** wire enterprise revenue and conversation-intelligence sources into Customer Requests, so a renewal risk surfaced in a sales call or a feature ask captured in a deal review enters Triage with provenance instead of dying in a CRM note.

The Method's posture does not change with scale: *customer feedback does not directly become scope* (P-22). A web-form submission, a Salesforce-attached request, a Gong-flagged ask — each lands in Triage as signal, and conversion to a project or issue stays an Operator decision in the relevant spoke. The larger the intake surface, the more load-bearing P-22 becomes: a public web form wired straight into a backlog is how a fifty-team workspace drowns.

When it pays for itself: the day intake outgrows the channels one Operator can watch — when requests arrive from a public form, a sales call, and a deal review faster than anyone can paste them in by hand.

What the tier does not design is the form taxonomy and the routing — which spoke owns which request type, which fields each form needs — the service-catalog work Business already flagged as engagement-shaped, now multiplied across the federation.

Agent guidance and Skills across the federation

Business introduced the agent surfaces (Chapter 8); Enterprise is where they scale across teams. **Shared agent guidance and Skills** let the hub author agent behavior once — how a Triage agent classifies, how a delegation packet is composed (P-15), how Validation evidence gets gathered — and publish it to every spoke, so the same standard runs everywhere instead of each Operator prompting from scratch. This is the community of practice expressed in agent terms: a Skill that proves itself in one spoke is promoted to the hub and inherited by all, exactly as a template or view is.

The Method's posture is unchanged and now matters more: agents are executors, not owners (P-14), and delegation still requires acceptance criteria and a validation command (P-15). A shared Skill that composes those criteria correctly is leverage across the whole federation; a shared Skill that skips them is a single point of failure replicated into every team.

When it pays for itself: the moment two spokes are prompting their agents to do the same job two different ways, and the hub realizes the behavior should be authored once and inherited, not re-invented per team.

What the tier does not build is the guidance itself — the prompts, the skill chains, the org-wide delegation protocol — the agent-layer engagement, now operating at the scale of the whole organization rather than one team.

Ceiling signal — when the method becomes the operating system

The lower tiers had a ceiling above them. Enterprise does not — there is no Tier 5. The signal here is not “you have outgrown the plan”; it is “the Method has

stopped being a practice and become infrastructure, and infrastructure needs an owner.” Three signals say you have crossed that line:

- **Drift outruns any one person’s ability to correct it.** New teams stand up workspaces weekly; each re-derives states, labels, and views, and half get it subtly wrong. The gap between the hub’s standard and the spokes’ reality widens every month because nobody owns closing it full-time.
- **Reporting only rolls up if the layers beneath are clean — and they are not.** The cross-team Dashboard Leadership opens is wrong, because three spokes have stale updates (P-19), two have a Done that means merged (P-6), and the evidence chain (P-28) is broken in places no single Operator can see from where they sit.
- **The org solves the gap with headcount instead of an operating model.** A VP of Operations, a TPM org, a platform team for “developer experience” — each hired to run the thing the Method describes, none handed an installed version of it, every one reinventing the federation’s shape from scratch.

When those land, the Method is no longer something a promoted Operator runs on the side. It is the organization’s operating system, and standing it up across the federation — the hub’s standards, the spokes’ overrides, the community of practice, the identity and audit controls, the cross-team reporting — is the work an engagement exists to do.

WHERE THIS GETS HARD

The organization that wants the federated model live across forty teams — one set of standards in the hub, justified local overrides in the spokes (P-18), a community of practice that actually moves templates between teams, SSO and SCIM satisfying the auditor, Dashboards Leadership trusts, agent guidance shared everywhere — has no single person with the authority, the bandwidth, and the artifact library to install all of it at once. The features ship the day you sign the Enterprise contract; the operating system takes a season and an owner.

Ravenhelm assesses your operating surface and installs the Method tuned to your team, your code host, and your reporting structure. nate@ravenhelm.co

Workflow Canon and the Validation Gate

INSTALL PATH	PRINCIPLES CITED	P-3	P-4	P-6	P-7	P-8	P-9	P-10	P-11	P-12
P-28										

It is 4:48 on a Friday. The Operator has already posted the weekly summary to Leadership: nine issues Done this sprint, two slipping into next week, one still blocked on a vendor. At 5:11 a customer pings in shared Slack — the export feature that shipped Tuesday returns an empty CSV. At 5:23, monitoring finally catches up: a migration ran on staging but never on production, so a different feature has been silently writing to a column that does not exist. By 5:40 the Operator is re-reading the Friday summary and counting which of those nine “Done” issues actually shipped working code to customers. The answer is six. The other three were merged, closed, and counted. Nobody looked at them after the PR went green.

The Method’s workflow exists to make that Friday impossible. It has exactly nine states. The one between merge and Done — Validation — is the load-bearing piece. The rest of this chapter argues that point and installs it.

The nine states

The Method’s workflow is canonized at nine states (P-10). Each has an entry condition, an exit condition, and a smell that signals misuse.

Triage. Unprocessed intake. Anything entering the workspace from a Slack ask, a Sentry alert, a customer request, an engineer’s thought, or the Operator’s own brain lands here first. Exit is classification: accepted, merged with another issue, sent to docs, escalated, or canceled. Smell: items aging past seven days untouched.

Backlog. Accepted but not scheduled. The work is real, it has an owner-eventual, but no one is starting it this week. Exit is scoping. Smell: a Backlog item with no project (P-3) — orphans accumulate here.

Ready. Scoped and available for an assignee or an agent. Acceptance criteria are written. The estimate is reasonable. The issue is deliverable-sized (P-4). Exit is someone picking it up. Smell: a Ready item nobody picks up for two weeks — usually the scope is wrong.

In Progress. Someone is actively working. Exactly one assignee. A branch exists if it is code work. Exit is a PR, an artifact, or a decision posted. Smell: more than one issue In Progress per person at a time.

In Review. A PR, artifact, or decision is awaiting review. Entry is automated when the code host opens a PR linked to the issue (Chapter 12). Exit is review approval and merge. Smell: an In Review item older than three working days — the reviewer is overloaded or the PR is too large (P-12).

Validation. Merged or delivered work still needs runtime, browser, release, infrastructure, or human-acceptance evidence (P-6). This is where most workspaces leak. Entry is merge or delivery; exit is evidence posted to the issue and accepted by the Operator. The rest of this chapter is mostly about this state.

Blocked. Stuck on something external; the blocker is named, in the issue, in a comment, with a date. “Blocked” without a named blocker is a parking lot, not a state. Exit is the blocker resolving or the issue being canceled. Smell: a Blocked item with no blocker named — almost always means the assignee has lost interest and needs the Operator’s intervention.

Done. Complete with evidence (P-7). The artifact link is in the issue. The evidence matches the acceptance criteria. Someone other than the assignee has confirmed, when human acceptance applies. Entry is the Operator (or, on higher tiers, an agent surface) accepting the Validation evidence. There is no exit; Done is terminal.

Canceled. No longer relevant; the reason is named. “We decided not to,” “the customer withdrew,” “superseded by issue X.” Cancellation without a reason is a state lie. Like Done, Canceled is terminal.

That is the canon. Nine states, in flow order. The reader’s existing workflow may have eleven or thirteen — most do. The Method’s first install act is to collapse it to nine.

Why Validation is load-bearing

Most workspaces operate on six states: Triage, Backlog, In Progress, In Review, Done, Canceled. PR merges, issue closes, sprint report counts it, the Operator moves on. This is the workflow that produced the 4:48 Friday at the top of this chapter.

The Method draws a hard line between two things that look the same and are not. **Merged code is implemented. Code that has passed evidence is trusted.** Only trusted code becomes Done (P-6). Validation is the named state where the difference lives.

The cost of skipping Validation compounds in three directions. First, the Operator's Done count becomes unreliable. "Nine issues Done this sprint" means nothing if three of them shipped broken. Worse, the Operator does not know which three. The number is theater (P-28). Second, the customer-facing surface develops a quiet backlog of half-shipped features — code merged, migration forgotten, flag not flipped, deploy never pushed to the customer-visible cluster. The team thinks it shipped. The customer does not see the change. The gap between team-reality and customer-reality grows until someone in a Slack DM names it. Third, Leadership loses faith in the rollup. The monthly initiative health report (Chapter 13) is built from project updates, which are built from issue counts. If issue counts lie, the entire reporting chain collapses into theater (P-28). The Operator becomes the person who keeps having to retract the Friday update.

Validation is the state that fixes all three at once. It is a holding pen with one rule: do not enter Done without evidence. The evidence is not the Operator's word; it is an artifact in a system the Operator cannot fabricate (P-8). The code host says the PR is merged. The deploy log says the binary is running. The screenshot shows the customer-facing URL renders. The Terraform plan says the resource exists. The customer reply says they tried it. Until one of those exists and matches what was promised, the issue stays in Validation.

This is the difference between a workspace that ships and a workspace that ships broken software with confident summaries.

AUGMENTATION SURFACE • Evidence Reconciliation

Where an agent layer makes this work better: scanning every issue in Validation on a schedule, checking the linked PR, deploy pipeline, release artifact, or browser surface for the expected evidence type, posting the evidence as a comment when it is findable, and escalating to the Operator only when it is not. The Validation queue stops being a queue the Operator clears manually and becomes a queue the Operator only sees when judgment is required.

What the PM keeps: the call on whether the evidence is actually sufficient — the human-acceptance test, the “is this really what we promised” judgment, the decision to push back when a screenshot is technically present but shows the wrong state.

The five evidence types

Not all Validation is the same. The Method recognizes five evidence types, each appropriate to a different kind of work. The Operator (and any agent layer assisting) needs to know which type the issue requires before it enters Validation, not after.

Runtime evidence — the code is running where it should be. The deploy pipeline log shows green. A log line confirms startup. A smoke test returns the expected payload. Use for backend changes, scheduled jobs, infrastructure services, anything where “the binary is up” is the question. Example artifact: a link to the deploy job output, plus a paste of the relevant log line.

Browser evidence — the user-facing surface is reachable and renders correctly. A screenshot of the page in the deployed environment. A short Loom of the new flow. A browser session log for an instrumented test. Use for any change a customer or internal user sees. Example artifact: a screenshot annotated with the URL bar visible, so the environment is unambiguous.

Release evidence — the work is in the release artifact customers actually receive. The release tag exists. The changelog entry is present. The app-store version is approved and rolled out. The container image is tagged and pulled by the production cluster. Use whenever merge to main is not the same as customer availability (P-9). Example artifact: a link to the release in Linear Releases or the equivalent code-host release page.

Infrastructure evidence — the underlying resources are in the expected state. A Terraform plan showing zero drift. A `kubectl get pods` output showing the expected count and image tag. A secret rotation log entry. Use for platform, capa-

city, security, and compliance work. Example artifact: a paste of the relevant command output, with the command itself included for reproducibility.

Human acceptance — someone other than the assignee has confirmed the work meets the acceptance criteria. The customer replied “this fixed it.” A UAT signoff comment from the requester. The Operator’s own acceptance comment after walking through the feature. Use whenever the question is “does this actually solve the problem we agreed to solve,” which is most non-trivial work. Never the assignee’s own assertion — that is not evidence, it is restatement.

For most issues, one evidence type is enough. For production work that ships to customers, the Method requires two: either runtime evidence plus browser evidence, or a deploy log plus a human-acceptance comment (P-9). The combination matters. A runtime green without a browser check has missed a frontend that points at the wrong API. A browser check without a deploy log has confirmed staging while production is still on the old binary. Two evidence types in production work is the cheapest insurance against the 4:48 Friday.

The three-person team flow

The Method’s flow for a 3-person team — Operator, Engineer A, Engineer B — is concrete enough to install on a Monday. Here is the lifecycle of a single deliverable.

The Operator and Engineer A scope the issue together in a fifteen-minute conversation. Acceptance criteria go into the issue body. The estimate is reasonable. The issue is deliverable-sized — single outcome, two weeks or less to verify, one owner (P-4). If the acceptance criteria run past five items, the issue splits into a parent with sub-issues (P-12). State moves from Backlog to Ready.

Engineer A picks the issue up Monday morning. The code-host integration (Chapter 12 owns the mechanics) creates a branch named with the issue ID, and the state moves to In Progress. Engineer A is the assignee. The branch name carries the issue ID so the code host can match commits and PRs back to Linear.

Engineer A opens a PR on Wednesday. The Linear-GitHub integration moves the issue to In Review automatically — on the GitLab equivalent, the merge-request integration does the same. Engineer B reviews. On a 3-person team, the reviewer pool is one person; this means review is fast, not optional. If Engineer B is heads-down on a separate issue, the Operator owns the call on whether the PR waits.

The PR merges Thursday. **Here the integration does not move the issue to Done.** It moves it to Validation (Chapter 12 covers the configuration). This transition is automated by the code-host integration (available on Free), or run by hand where it isn't wired. Either way, the rule is the same: merge does not equal Done.

Engineer A posts evidence in a Validation comment. For a backend change, a link to the deploy log plus the relevant startup line. For a UI change, a screenshot of the deployed environment with the URL visible. For an infrastructure change, the Terraform plan output. The evidence type was decided when the issue was scoped, not invented now.

The Operator confirms the evidence matches the acceptance criteria. Not the Operator's vibe — the literal acceptance criteria written in the issue body when the work was scoped. If it matches, the Operator moves the issue to Done. If it does not, the Operator comments on the gap and the issue stays in Validation. The assignee does not move their own work to Done.

If the change is production and customer-visible, the Operator waits for release evidence — the Linear Releases entry, or the equivalent on the code host — before Done (P-9). For a 3-person team this is usually a one-day wait, not a one-week wait. The cost is small. The cost of skipping it is the 4:48 Friday.

The Leadership Team never sees any of this. They see the Friday summary and the monthly rollup. The reason those rollups are trustworthy is that the issue counts behind them mean what they say.

Nine states is the contract

When someone — a new engineer, an experienced PM joining the team, a vendor consultant — proposes a tenth state, the answer is no. The most common proposals are “In QA,” “Pending Approval,” “Design Review,” and “Soft Launch.” Each one feels reasonable. Each one is a `stage:*` label, not a state (P-11).

“In QA” is `stage:qa` on an issue that is In Progress or In Review. “Pending Approval” is `stage:awaiting-approval` on an issue in Validation. “Design Review” is `stage:design-review` on an issue in In Review. “Soft Launch” is `stage:soft-launch` on an issue in Validation, waiting for full release evidence before Done. Labels carry the variation; the canon stays at nine (P-11).

This is doctrine, not preference. The reason is brutal: a workflow with thirteen states has thirteen places work can stall invisibly. A workflow with nine has nine. The Method's nine are the smallest set that preserves the Validation dis-

tion, which is the only state difference that actually matters for the question “did this ship?” Everything else is a label (P-11), a comment, or a project milestone.

The canon is nine. Hold it.

WHERE THIS GETS HARD

The Operator’s team does not own production. A separate platform team controls the deploy, the cluster, and the release pipeline. Getting a deploy-log link on every customer-visible Validation issue requires the platform team’s cooperation, which they have no incentive to provide — their on-call rotation does not care about your Linear hygiene, and the link in their CI system requires permissions you do not have.

Ravenhelm assesses your operating surface and installs the Method tuned to your team, your code host, and your reporting structure. nate@ravenhelm.co

Cycles, Roadmaps & Cadence

PRINCIPLES CITED P-2 P-3 P-4 P-6 P-10 P-13 P-19 P-21 P-24 P-26 P-27
P-28

Engineer B came from a shop that ran two-week sprints, and in the second week asks the question the Operator has been avoiding: are we doing sprints or not? That same week Leadership asks for “the roadmap” — not this week’s board, the next two quarters, on one page, by Thursday. The Operator has neither answer cleanly. They tried cycles once, early: turned them on, watched Linear produce a velocity chart, and three cycles later had a pile of rolled-over issues, a number nobody trusted, and a creeping sense the team now served the cycle instead of the other way around — so they turned it back off. And the roadmap they sent Leadership last quarter was a screenshot of the project board, already stale by the time it was pasted into the deck; the first question in the meeting was about a project that had shipped a week earlier.

Two real gaps, and the Method has so far been silent on both. The first is execution rhythm: does the team commit work in fixed batches or flow it continuously? The second is planning horizon: how does the team show the next two quarters without lying? Linear answers both on Free — Cycles and the timeline roadmap are free primitives. The Method’s job is to say when to reach for each, and, more often, when not to.

Start from what the Method already runs on. The operating rhythm is cadence: a weekly project update (P-19), a biweekly initiative update, a monthly operating review (P-21, Chapter 13). That cadence is the spine, and it works whether or not the team ever turns cycles on.

Cycles are an optional execution-rhythm overlay. The roadmap is a planning horizon. Neither replaces the cadence; both sit on top of it.

Cadence is the spine; cycles are an overlay

A cycle is Linear's name for a sprint: a fixed time box — usually two weeks — into which the team commits a batch of issues, with a start, an end, a carried-forward set, and a velocity number that accumulates across cycles. Linear makes them nearly free to turn on, which is exactly the risk.

The Method's default is cadence-driven, not cycle-driven. The weekly project update (P-19) already tells the team and Leadership what shipped, what is in flight, and what is blocked; the milestones on a project (P-3) already mark the meaningful execution boundaries. For many 3-person teams that is the whole rhythm they need, and a cycle on top of it is a second clock measuring the same time. The overbuild test (P-24) is the filter: a cycle has to help the Operator decide faster or surface risk earlier, or it is ceremony with a burndown chart attached.

When to turn cycles on

Turn cycles **on** when the team benefits from a fixed batch and a rollover review. The signals are concrete:

- The work decomposes cleanly into two-week chunks.
- The team wants a heartbeat to commit against rather than an open backlog.
- The end-of-cycle review — what we committed, what carried over, why — would tell the Operator something the weekly update doesn't.

A team building steadily toward a known scope, where “are we moving fast enough” is a live question, is a team a cycle serves.

Leave cycles **off** when the team runs on continuous flow. The signals are the inverse:

- The work is interrupt-heavy — ops, support, on-call — and arrives unpredictably.
- Deliverables refuse to batch into two-week boxes; the plan was always going to bend to the next interrupt.
- The project update and the milestone already supply the rhythm; a cycle just adds a boundary the work keeps crossing and a rollover pile that means nothing.

Continuous-flow teams are driven by project updates and milestones, not by a sprint boundary.

Two hard warnings. First, **do not turn cycles on before the workflow canon is stable** (Chapter 10). A cycle measures the throughput of your Done column; if Done still means merged rather than validated (P-6), the velocity number just measures the lie faster. Stabilize the nine states (P-10) and the Validation gate first; add the cycle only once Done is trustworthy.

Second, **a cycle is not a roadmap**. A cycle is a two-week execution box; a roadmap is a multi-quarter horizon. Teams that run the current cycle board as their plan are showing Leadership two weeks of detail and calling it strategy. The horizon lives in initiatives and the timeline, covered below.

Running a cycle: capacity and the seventy-percent rule

Once cycles are on, the discipline lives at the cycle boundary. Before a cycle starts, the team enters capacity — estimates on the candidate issues, and an honest read of how much each person can take given the days they are actually present. Linear sums the load; the Operator commits the cycle.

This is where the Method resolves a tension. Principle P-26 says sprint-or-cycle planning commits seventy percent of expected capacity, holding thirty percent for unplanned work. Read literally against a Method whose default has no cycles, P-26 looks like it assumes a ceremony the Method doesn't require. It doesn't. **P-26 is a capacity rule, not a cycle rule.**

The seventy-percent commitment is where the rule lives when cycles are on, and it is the most important number in cycle planning: a team that commits its full estimated capacity slips every cycle and learns nothing, because the explanation is always “something came up” (P-26). When cycles are off, the same thirty-percent reserve still holds — just at the weekly-commitment level instead of the cycle boundary. The Operator does not load Engineer A and Engineer B to the brim for the week either. Cycles give P-26 a place to be enforced; they are not what makes it true.

Cycle automations carry the mechanics. Linear can auto-add issues to the current cycle, roll unfinished issues forward to the next one, and run a cooldown between cycles. Let the automations do the moving; the Operator's job is the rollover *review*, not the rollover itself. The review asks one question of every carried-forward issue — why didn't this finish?

- It was bigger than estimated — a scoping miss (P-4).
- The team was interrupted — the thirty percent was real (P-26).

- It was never actually started — a prioritization problem the Operator owns (P-13).

The pile is data; the review is where it becomes a decision. The cycle board, meanwhile, is still not the plan you show Leadership — that is the roadmap, which is the rest of this chapter.

Roadmaps: the timeline over initiatives and projects

The roadmap answers Leadership's other question — the next two quarters on one page. In Linear it is the timeline view: date-bearing projects laid horizontally on a calendar, swimlaned by initiative.

Each project carries a start date and a target date (P-3); each initiative is the durable outcome those projects ladder up to (P-2). The roadmap is the objective layer rendered against time — initiatives as the rows, projects as the bars, the quarter as the horizon.

Two things the timeline shows that no board can. First, **sequence across initiatives**: when a project under Platform Reliability has to finish before a project under New Product Bring-Up can start, the dependency is a native blocking relation (a Free primitive) and the timeline draws it as a line between bars. The Operator sees the cross-initiative dependency that a per-team board hides. Second, **horizon honesty**: a project bar that runs past the quarter edge is a visible claim the Operator has to defend, where a board just shows “In Progress” with no end in sight.

The roadmap feeds the monthly operating review (Chapter 13). The initiative rol-lup the Operator presents to Leadership is the same set of bars, compressed to health and trend (P-21). The roadmap is not a separate artifact maintained alongside the reporting stack; it is the visual the reporting stack reads from.

Plan-of-Record vs. What-If

The roadmap is only trustworthy if there is exactly one of it. The discipline is to keep a single committed roadmap — the **Plan-of-Record** — separate from every scenario the Operator wants to explore. The Plan-of-Record is what the team executes against and what Leadership reads (P-27); it changes deliberately, with a reason, never casually because someone dragged a bar in a planning meeting.

What-if planning is real and necessary — what if we pull the integration project forward a month, what if a fifth engineer joins next quarter, what if we drop the

compliance work to ship revenue sooner. That work belongs in a scenario kept visibly separate: a duplicated draft timeline, a planning document, a clearly-labeled what-if view — never edits to the committed plan. The moment the team can't tell whether a date is a commitment or a hypothesis, the roadmap stops being something Leadership can trust, and the evidence chain (P-28) breaks at the top: an initiative rollup built on dates nobody actually committed to is theater.

One Plan-of-Record; as many what-ifs as you like, all of them clearly not it. Promoting a what-if into the Plan-of-Record is the Operator's call, and it is a commitment (P-13), not a drag-and-drop.

AUGMENTATION SURFACE · Capacity and Scenario Modeling

Where an agent layer makes this work better: forecasting a cycle's realistic load from each person's actual availability and historical carry-over before the Operator commits it (P-26); generating what-if timelines on request — pull this project forward, add an engineer next quarter, drop that initiative — as throwaway scenarios that never touch the Plan-of-Record; and watching the committed roadmap for drift, flagging the project whose target date has quietly slipped past its initiative's horizon before the monthly review surfaces it.

What the PM keeps: the commit itself — which cycle to start, which what-if becomes the plan, and which slipping date is a real risk to escalate versus a bar to redraw (P-13).

Rhythm and horizon, not new bureaucracy

Put the three together and nothing new is added to the operating model. The cadence is the rhythm the Operator already runs:

- **Daily** — clear Blocked and the Operator's own queue.
- **Weekly** — the project update (P-19).
- **Biweekly** — the initiative update.
- **Monthly** — the operating review (P-21).

The roadmap is the horizon that rhythm reports against — the quarter-plus view the monthly review rolls up to. A cycle, if the team adopts one, is a sub-rhythm that sits *under* the weekly update: a two-week execution heartbeat, not a replacement for it. The Operator still writes the Friday update whether or not a cycle is running; the cycle just gives the team a batch boundary inside the week-by-week flow.

That is the whole relationship. Cadence is rhythm. The roadmap is horizon. Cycles are an optional heartbeat under the rhythm. A team that mistakes any one for another — runs the cycle board as its roadmap, treats the roadmap as a weekly status, or bolts a cycle onto continuous-flow work that doesn't want one — has bought bureaucracy. A team that keeps the three in their lanes has rhythm and horizon for free, on the free tier, and answers Engineer B's sprint question and Leadership's roadmap question from the same operating model it was already running.

WHERE THIS GETS HARD

The roadmap Leadership wants and the roadmap the work supports diverge the moment a date becomes a promise to someone outside the team — a customer commitment, a board deadline, a launch already announced. Now the Plan-of-Record is under pressure to show the date Leadership sold, not the date the projects support, and the Operator is asked to drag a bar to make a slide true. Holding the line — committed dates reflect committed work, what-ifs stay labeled — is a credibility fight the Operator rarely has the seniority to win alone.

Ravenhelm assesses your operating surface and installs the Method tuned to your team, your code host, and your reporting structure. nate@ravenhelm.co

Code Host Integration: GitHub Primary, GitLab Parallel

PRINCIPLES CITED P-6 P-7 P-8 P-9 P-18 P-23 P-24 P-25 P-29

The Operator installed the Validation state two weeks ago (Chapter 10). It worked. Issues stopped flipping to Done the second a PR went green. Friday summaries match Friday reality again. But there is a new problem, and it shows up Sunday night. The Operator is in Linear with a coffee and a list of thirty-one issues that need state changes — fifteen PRs opened this week that still say In Progress, eleven merged PRs sitting in In Review because nobody dragged them to Validation, four shipped releases whose issues are still in Validation, one ancient In Progress that turned out to be a PR Engineer A merged a month ago. The Operator is updating Linear by hand from the code host’s “Pull Requests” tab. This is the exact failure mode P-25 names: a convention you cannot verify programmatically decays into folklore.

This chapter wires Linear to the code host so the convention enforces itself. The code host and Linear are the same system from the Operator’s point of view: the code host is the source of truth for implementation (P-8); Linear is the source of truth for status (P-23). When they are wired correctly, every PR event in the code host produces the right state change in Linear, and the Operator’s Sunday-night queue collapses from thirty-one items to the handful of issues that actually require judgment.

GitHub is the primary example. GitLab gets equal treatment inline — every mechanic here works in either code host with different plumbing and the same Linear-side conventions. The Method is code-host-agnostic on purpose: the 3-person team is rarely mono-host, and the Operator should not have to relearn the model when an engineer ships from a different repo.

Branch and PR conventions

The first mechanic is the convention that lets the code host and Linear find each other at all. Every branch carries the Linear issue ID. Every PR title prefixes the issue ID in brackets. Every commit body, when it matters, references the issue. The integrations look for these strings; without them, no automation fires.

The pattern is `<issue-id>-<short-slug>` for branches and `[<issue-id>] <description>` for PR titles. Engineer A, working on `web-frontend` (a GitHub repo), opens a branch called `ENG-42-rate-limiter` and a pull request titled `[ENG-42] Rate-limit the public search endpoint`. Engineer B, working on `api-service` (a GitLab repo), opens a branch called `ENG-58-token-rotation` and a merge request titled `[ENG-58] Rotate service account tokens nightly`. Same convention, different code host. Linear’s integration on each side picks up the issue ID from the branch and the title; the linked PR or MR appears on the issue automatically.

CONVENTION	GITHUB EXAMPLE	GITLAB EQUIVALENT
Branch name with issue ID	<code>ENG-42-rate-limiter</code>	<code>ENG-58-token-rotation</code>
PR / MR title prefix	<code>[ENG-42] Rate-limit search</code>	<code>[ENG-58] Rotate tokens nightly</code>
Commit body reference	Refs <code>ENG-42</code> in commit message	Refs <code>ENG-58</code> in commit message
Linking trigger	Linear GitHub App parses branch + PR title	Linear GitLab integration parses branch + MR title; or webhook posts to Linear API

The Operator publishes this convention as a one-page repo-level guideline (**P-18** — closer-to-the-work guidance wins) and adds a PR-template stub that prompts engineers to fill in the issue ID. The verification check is cheap: a workspace-wide Linear view filtered to In Progress issues with no linked PR after two days. That is the only enforcement that survives (**P-25**).

PR opened triggers In Review

The second mechanic is the simplest automation. When a PR or MR opens against the main branch, the linked Linear issue moves from In Progress to In Review. The Operator does not move it. The engineer does not move it. The integration moves it, every time, the moment the PR opens.

On GitHub, this is a built-in capability of the Linear GitHub integration. The Operator opens team settings, finds the GitHub section, selects “PR opened” → “In Review” for the team. Done. Engineer A’s ENG-42 PR opens on `web-frontend` and the issue transitions before the Operator’s coffee cools.

On GitLab, the path is slightly different. Linear’s GitLab integration covers MR linking and basic status sync; for finer control, the Operator installs a project-level GitLab webhook on the MR-events channel that posts to Linear’s API with the desired transition. Engineer B’s ENG-58 MR opens on `api-service` and the same transition fires — In Progress to In Review — via the webhook. The reader-visible behavior is identical; the configuration differs by one settings panel.

The Operator should not skip this step waiting for the “perfect” automation. PR-opened → In Review is the cheapest, highest-signal transition in the entire model. It pays for itself in the first day.

PR merged triggers Validation — not Done

The third mechanic is where most code-host integrations get this wrong. The default behavior on most platforms — and the tempting setting on Linear’s side — is to move issues to Done on PR merge. The Method explicitly refuses this. Merged code is implemented; only evidence makes it trusted (P-6). The auto-move on merge goes to Validation. Done remains a human decision until release-driven automation is installed (next section).

On GitHub, the Linear GitHub integration exposes per-team “PR merged” automation. The Operator selects Validation, not Done. Engineer A merges ENG-42; the issue moves to Validation; the merged PR URL lands on the issue automatically, which satisfies the evidence-pointer requirement (P-7) for implementation but not for delivery.

On GitLab, the same outcome comes from a webhook listening for `merge_request` events with `action: merge`, posting an issue-state update to the Linear API. Engineer B merges ENG-58; the webhook fires; the issue moves to Validation. Same Linear-side behavior, different plumbing.

The temptation to “just enable PR-merged → Done” is the single most common installation mistake. Every issue that ships to customers requires release evidence before Done (P-9). Every issue that runs in production requires runtime or browser evidence. If the integration closes those issues automatically, the Validation state collapses into theater and the Method’s load-bearing principle (P-6) is silently disabled. The Operator must hold this line.

AUGMENTATION SURFACE · Integration Drift Detection

Where an agent layer makes this work better: scanning the workspace daily for state changes that *should* have happened but didn't — PRs marked merged in the code host with their Linear issue still in In Review, releases shipped to production with their issues still in Validation, repos that have stopped producing linked PRs at all (a sign the convention has decayed or the integration broke). The agent checks webhook delivery logs on the code host, surfaces failed deliveries, and posts a daily integrity summary so the Operator finds out about drift before the Friday update.

What the PM keeps: deciding whether a stuck transition is a tooling problem (re-deliver the webhook, fix the branch name) or a missing-evidence problem (the issue genuinely lacks proof and is staying in Validation for the right reason).

Release events trigger Done

The fourth mechanic closes the loop for production work. When a release ships, every issue in that release's scope moves from Validation to Done. The release event is the evidence (P-9); the integration posts the artifact link as a Done comment automatically.

On GitHub, the Operator wires a release workflow in GitHub Actions. The workflow runs on the `release: published` event, calls Linear's Releases API (or uses the published `linear-release-action`), passes the list of merged PRs since the previous tag, and tells Linear to associate those PRs' linked issues with the release. The Linear Releases pipeline is configured to move associated issues to Done on release publication. Engineer A's `ENG-42` is included in the next `webfrontend` release; the action fires; `ENG-42` transitions to Done with a link to the release page as the evidence artifact.

On GitLab, the equivalent is a CI release job. The `.gitlab-ci.yml` includes a `release` stage that runs on tag pipelines, calls the GitLab Releases API to publish the release notes, and posts to the Linear Releases pipeline via the same Linear API as the GitHub side (Linear does not care which CI system fired the call). Engineer B's `ENG-58` ships in the next `api-service` tag; the CI job fires; `ENG-58` transitions to Done.

PIPELINE	WATCHED EVENT (GITHUB)	WATCHED EVENT (GITLAB)
Production release	<code>release: published</code> workflow in GitHub Actions	<code>release</code> job in GitLab CI on tag pipeline

PIPELINE	WATCHED EVENT (GITHUB)	WATCHED EVENT (GITLAB)
Staging release (optional)	Workflow on push to staging branch or manual dispatch	CI job on staging environment deployment

Two warnings on releases. First, install release-driven Done one surface at a time. The Operator picks the highest-volume customer-facing repo, wires the pipeline, lets it run for a week or two with the Operator double-checking each release. Only after that flow is green does the second surface get the same treatment. Second, do not exceed roughly six Releases pipelines in the workspace (P-24). Pipelines are operational surface; each one needs a watch and a refresh. More pipelines means more decay (P-25). If the team ships from twelve repos, the Operator picks the six that most need release-gated Done — usually the customer-facing surfaces — and lets the rest stay in manual Done.

Naming releases: Semantic Versioning

The release event that moves issues to Done fires against a release, and a release needs a name. The Method’s convention is not negotiable: releases are named with Semantic Versioning — MAJOR.MINOR.PATCH — and nothing else (P-29). A version like 2.4.1 is not a label the Operator picks on release day; it is a fact the commits already decided.

The mapping is the whole rule. A breaking change bumps MAJOR (2.4.1 → 3.0.0); a backward-compatible feature bumps MINOR (2.4.1 → 2.5.0); a backward-compatible fix bumps PATCH (2.4.1 → 2.4.2). An engineer who reads 2.5.0 → 2.5.1 knows a fix shipped and nothing broke; one who reads 2.5.1 → 3.0.0 knows to check the changelog before upgrading. The version number carries information — that is the point of naming this way instead of any other.

Neither the Operator nor the engineer computes the bump by hand. A tool derives it: semantic-release (or release-please) reads the Conventional Commits since the last tag — fix: maps to PATCH, feat: to MINOR, a feat!: or BREAKING CHANGE: footer to MAJOR — then picks the bump, writes the changelog, cuts the tag, and publishes the release. The version, the changelog, and the tag are a byproduct of commit discipline, not a decision anyone makes on release day. That feeds the previous section directly: the release: published event needs a release to publish, and now the release names itself from the commits Engineer A and Engineer B already wrote.

This is why SemVer lives in the same chapter as release-driven Done. The Linear Releases pipeline publishes a SemVer tag and moves the issues whose PRs

landed between the previous tag and this one from Validation to Done (P-9). Consistent naming is what makes that association legible: the pipeline records that `ENG-42` shipped in `2.5.0`, and the Operator can answer the only release question Leadership ever asks — which version shipped what. Chapter 13’s reporting rollup consumes that answer, and release health there is only as readable as the version names beneath it. A workspace that tags releases `final`, `final-2`, and `prod-hotfix-friday` has release automation that runs and a release history nobody can read.

The 3-person team installs both

This is what makes the chapter concrete. The Operator’s team ships from two code hosts. Engineer A owns `web-frontend` and pushes to GitHub; Engineer B owns `api-service` and pushes to GitLab. The shared `infra` repo, where both engineers commit occasionally, also lives on GitHub. The Method’s installation on the team is mixed by design: the Operator does not migrate either repo to standardize, because the cost of migration is higher than the cost of running two integrations.

What the Operator installs is identical on both sides at the Linear layer: the same nine-state workflow, the same Validation gate, the same auto-move on PR merge to Validation (not Done), the same release-driven Done for the production surface. What differs is one configuration panel per host. The Linear GitHub App is installed once for `web-frontend` and `infra`; the Linear GitLab integration plus one MR webhook is installed once for `api-service`. The Operator writes the conventions once and publishes them at the workspace level; each engineer follows the same branch naming, the same PR title prefix, the same commit references. The Leadership Team consumes the same project updates regardless of which host the underlying PRs live on, because by the time the rollup reaches them (Chapter 13) the host is invisible — only the Linear state remains.

This is the proof that the Method is code-host-agnostic. The conventions are at the Linear layer; the plumbing differs; the reader-visible behavior is the same.

Self-hosted forges — the webhook bridge

Everything above assumes a code host Linear drives natively — GitHub through its App, GitLab through its integration. Some teams do not have one. A team on a self-hosted forge — Forgejo, Gitea, or a Bitbucket instance Linear does not natively integrate with — gets no PR-opened transition, no PR-merged transition,

and no linked PR on the issue. The branch-and-title convention still holds, but nothing is listening for it, and the Operator is back to the Sunday-night manual queue with no integration to install.

The Method's answer is a webhook bridge, deliberately small. The forge fires a webhook on PR merge; a tiny receiver parses the payload, looks for a magic word in the PR title or body — Fixes <ISSUE-ID> or Closes <ISSUE-ID>, case-insensitive — and drives Linear through its GraphQL API. Three calls do the whole job. Resolve the human issue ID (TEAM-123) to the issue's UUID, because the API keys on the UUID, not on the identifier a person reads. Move the issue to Validation — not Done, the same line the native integrations hold (P-8: the merge proves implementation; P-9: shipping still needs release evidence). Attach the PR URL back onto the issue as the evidence pointer. The bridge reproduces what the GitHub App does for free, and nothing more.

Migrate onto it carefully. Do not cut the native path over on a hope: keep a GitHub or GitLab mirror of the repository while the team moves, so the native PR→Linear automation keeps firing off the mirror until the bridge has been trusted for a week or two on live merges. Mirror-back is the safety net — if the receiver drops an event, the integration on the mirror still moves the issue, and the Operator loses nothing while the bridge earns confidence. Retire the mirror only once the bridge has been green long enough that the Operator has stopped checking it by hand.

One authentication detail accounts for most bridges that fail on the first try. Linear's GraphQL endpoint authenticates a personal API key sent raw in the header — Authorization: <api-key>, with no Bearer prefix. A developer reaching for OAuth muscle memory writes Authorization: Bearer <api-key>, gets a 401, and loses an afternoon to it. The key goes in raw.

A bridge is a build-once, verify-forever surface (P-25): checked once against a real merge, and after that every merge lands its issue in Validation without the Operator watching. But it is also code the team now owns, so it lives under the overbuild ceiling (P-24) — one receiver, one magic-word rule, one transition, which a 3-person team can keep running. A bridge that grows into bidirectional sync (P-23), per-repo state machines, or a second source of truth has failed the test; at that point the honest move is to migrate the repo to a host Linear drives natively, not to grow the bridge.

Install order

Phase the install. Do not turn on every automation in one afternoon.

1. **Install the integration.** GitHub App on the GitHub organization, GitLab integration on the GitLab group. Connect each engineer's personal account on each side.
2. **Enable PR linking and the In Review transition per team.** Configure PR-opened → In Review on the Linear side for each team that owns code work. On GitLab, install the MR-events webhook now.
3. **Confirm one issue end-to-end.** Pick an in-flight issue. Watch the PR open; confirm In Review fires. Watch the PR merge; confirm Validation fires. Done still moves by hand. Hold here for one to two weeks. The Operator is verifying the convention enforces itself before adding more automation.
4. **Install Releases pipelines for the production surface.** After the manual-Validation flow has been green for a week or two, wire the GitHub Actions release workflow (or GitLab CI release job) for the highest-volume customer-facing repo. Configure the Linear Releases pipeline to auto-move associated issues from Validation to Done on release publication. Done becomes automatic for that surface only.
5. **Replicate to additional surfaces.** Once the first surface has a green release flow for one to two weeks, replicate to the next production surface. Cap at roughly six pipelines workspace-wide (P-24).

What not to do

Two patterns destroy this chapter's value if installed.

Do not sync GitHub Issues (or GitLab Issues) into Linear bidirectionally. Linear is the system of record (P-23); the code host's issue tracker is a duplicate. One-way webhooks from the code host into Linear are fine — Sentry alert into Triage, dependabot finding into Triage, a public GitHub Issue routed into Triage for the Operator's review. Two-way sync is debt. Drift between Linear and the code host's issue tracker becomes impossible to debug, the Operator ends up updating both, and the verification check (P-25) collapses because there is no single source of truth to verify against.

Do not install more than roughly six Releases pipelines. Each pipeline is a watched event, a CI job, a webhook, and a Done-transition rule. Each one needs to be audited quarterly. Six is the rough capacity ceiling for a 3-person team (P-24); above that, pipelines start failing silently and the Operator stops trusting

the Done count. If twelve repos need release-gated Done, the Operator’s next decision is not “more pipelines” — it is “fewer surfaces” or “a separate orchestration layer.”

WHERE THIS GETS HARD

When the release flow involves multiple repos coordinated through a separate orchestration layer (Argo, Spinnaker, an internal CD platform), or when release gates legitimately span days because of phased rollouts, canary windows, or compliance signoffs, the simple “release event → Done” automation stops fitting. The release event happens at a layer the code host does not own, and the issues need to wait for a signal that lives somewhere else entirely. At that point the Operator is designing custom plumbing — release pipelines that listen to the deployment system, not the code host — and the chapter’s defaults stop applying.

Ravenhelm assesses your operating surface and installs the Method tuned to your team, your code host, and your reporting structure. nate@ravenhelm.co

Reporting Up: Project Updates, Initiative Health, and the Monthly Review

INSTALL PATH	PRINCIPLES CITED	P-2	P-6	P-7	P-8	P-19	P-21	P-25	P-27
P-28									

It is Thursday at 4:15 p.m. You have not started the leadership status note yet. Your engineering Slack channel needs a “this week so far” pin by end of day. The all-hands deck — the one Leadership wants Friday morning — has a slide titled “Platform Reliability — status?” and the slide is blank. The leadership Notion page wants a bulleted update by Monday and the bullets are supposed to follow last quarter’s format, which nobody remembers. You open three tabs, three editors, three different sets of headings, and you write what is functionally the same paragraph three times in three voices. The first version is too operational, the second is too vague, the third sounds like a press release. By Friday at noon you have produced three reports, none of which Leadership reads end-to-end, and none of which you can quote on Monday when someone asks you what changed last week. You did the work. The work did not compound.

The Method’s reporting layer exists to make that compounding happen. The Operator writes once, at the right altitude, in a fixed template, and the same content rolls up. Leadership reads summaries, not raw Linear (P-27). Your job is translation — from issues to project updates, from project updates to initiative updates, from initiative updates to the monthly review. Each layer compresses the one below it. The compression only works if the format below is stable enough to compress mechanically.

The three cadences and the nesting

Three rhythms hold the stack together. Weekly, every active project gets a project update (P-19). Biweekly, every active initiative gets an initiative update. Monthly, the initiative updates feed a 30-minute (or 5-minute-read) operating review for Leadership (P-21). Project → initiative → review. The Operator writes upward; Leadership reads downward and almost never reaches past the top.

The evidence chain (P-28) is what makes the rollup honest. Initiative health rolls up from project updates. Project updates roll up from issue evidence. Issue evidence rolls up from the code host — merged PRs, deploy logs, acceptance comments (P-7, P-8). Break any link and the chain collapses into theater: green initiative status that nobody can defend, monthly reviews that recite intentions, all-hands updates assembled from memory. The templates that follow are what keep the chain unbroken. They are not aesthetic choices. Each section forces the Operator to either point at evidence or admit there isn't any yet.

The weekly project update

Every active project gets an update every week. No exceptions for “nothing happened” weeks — write the update anyway, mark the items as carried over, and keep the cadence. A project without an update for seven days surfaces in the stale-projects view; that view is what makes the cadence enforceable rather than aspirational (P-19).

The Operator posts the update on Friday afternoon as a project update in Linear. The team is also a reader. Engineer A and Engineer B should glance at it before logging off Friday; the update tells them what shipped, what is in flight, and what is blocked going into Monday. It is not a Leadership-only artifact.

```
# [Project Name] — Week of [YYYY-MM-DD]

## Shipped this week
- [Issue title] — [one-line outcome]. [PR link or evidence link]
- [Issue title] — [one-line outcome]. [PR link or evidence link]

## In flight
- [Issue title] — In Progress. Owner: [name]. Expected: [date or milestone].
- [Issue title] — In Review. PR: [link]. Awaiting: [reviewer or check].
- [Issue title] — Validation. Evidence needed: [runtime smoke / browser proof / acceptance].

## Blocked
- [Issue title] — blocked on [specific blocker]. Unblocker: [name or system].
```

```

Age: [N days].

## Decisions needed
- [Question, one sentence]. Owner of the decision: [Operator or Leadership].
Decide by: [date].

## Next week
1. [Top priority]
2. [Second priority]
3. [Third priority, if any]

```

Three failures the template prevents. First, the “everything is fine” update — the one that says “good progress, no blockers” and elides the two issues quietly stuck in Blocked for nine days; the template forces a Blocked section, and an empty section is a claim the Operator has to be able to defend. Second, the wall-of-text update — five paragraphs of context with no actionable items; the template’s bullet structure makes that physically awkward to write. Third, the buried decision request — an Operator-needs-a-Leadership-call hidden in the middle of paragraph three; the Decisions section makes it the part Leadership can scan in 10 seconds.

AUGMENTATION SURFACE • Update Drafting

Where an agent layer makes this work better: pulling the project’s shipped issues (Done in the last 7 days), in-flight issues (In Progress, In Review, Validation), Blocked items with their blocker comments, and Waiting-on-Operator items with their decision deadlines into a first-draft project update every Friday morning — every section of the template pre-populated, every link in place, ready for the Operator to edit by lunch. The same pattern compounds upward: biweekly initiative drafts assembled from the project updates underneath them, monthly review drafts assembled from the most recent initiative updates. The drafting is the high-leverage part. Most of the Operator’s translation work becomes review and edit, not authorship.

What the PM keeps: the framing call (“this is the bet next cycle, here is why”), the risk call (what to flag to Leadership now and what to hold a week), and every decision request — the precise wording, the deadline, and who owns the decision.

The biweekly initiative update

Every two weeks, every active initiative gets an update. Initiatives are the objective layer (P-2) — durable outcomes that roll up projects, with target horizons no tighter than a quarter. The biweekly cadence is the compression step

between weekly project chatter and the monthly Leadership rollup. The Operator should not be writing initiative updates more often than biweekly; the underlying work does not move fast enough to justify weekly initiative-level prose, and writing it weekly produces noise that drowns the signal Leadership is supposed to see.

Use the generic initiative archetypes any 3-person team can adopt: Platform Reliability, Operator Experience, New Product Bring-Up, Compliance Readiness, First Revenue. Substitute whatever names your own initiatives carry; the template is identical.

```
# Initiative: [Initiative Name] – Update [YYYY-MM-DD]

### Objective health
**[Green | Yellow | Red]** – [one-line reason if not green]

### Key result status
| KR | Current | Target | Trend |
|---|---|---|---|
| [Metric or outcome] | [value] | [target] | [up / flat / down] |
| [Metric or outcome] | [value] | [target] | [up / flat / down] |
| [Metric or outcome] | [value] | [target] | [up / flat / down] |

### Narrative
[One paragraph. What changed this cycle. What is the bet for the next cycle. What is the one risk Leadership should know about – the one the Operator would otherwise want to hide.]

### Linked projects and current status
- [Project Name] – [On Track / At Risk / Blocked]. Latest update: [link]. One line: [what is happening].
- [Project Name] – [On Track / At Risk / Blocked]. Latest update: [link]. One line: [what is happening].
```

Failures the template prevents. First, the perpetually-green initiative — the one whose status has been green for nine biweeks because nobody wants to be the one to flip it; the Trend column on every KR forces a directional claim, and three flats in a row is itself a yellow signal. Second, the narrative as press release — the paragraph that lists accomplishments and omits the bet and the risk; the prompt explicitly asks for what the Operator would otherwise want to hide. Third, the initiative untethered from projects — health asserted without project evidence underneath; the Linked Projects section makes the rollup chain visible, and an initiative with no recent project updates beneath it is not actually being run.

The monthly operating review

At the end of each month, the Operator publishes a single document that consumes the most recent initiative updates and presents the state of the portfolio to Leadership. It is consumed in one of two ways: a 30-minute meeting where the Operator walks Leadership through the rollup, or a 5-minute read in advance with a 15-minute decision conversation after. Both should work from the same document. Leadership does not browse the workspace; this document is the workspace from their perspective (P-27).

```
# Monthly Operating Review – [Month YYYY]

## Initiative rollup
| Initiative | Health | KR summary | Trend |
|---|---|---|---|
| [Initiative Name] | [G/Y/R] | [1-line KR state] | [up / flat / down] |
| [Initiative Name] | [G/Y/R] | [1-line KR state] | [up / flat / down] |
| [Initiative Name] | [G/Y/R] | [1-line KR state] | [up / flat / down] |

## What shipped this month
[5–10 lines. The completed work that mattered, not every closed issue. Each line
names the outcome and links to the evidence – PR, release, acceptance artifact
(P-7).]

## Stale work
- **Projects without recent updates:** [list, with last-update date]. (P-19)
- **Triage items past SLA:** [count and oldest age]. (P-20)
- **Initiatives without movement:** [list, with last initiative update date].

## Decisions ahead
- [Decision Leadership will need to make]. Expected by: [date]. Why now: [one
line].
- [Decision Leadership will need to make]. Expected by: [date]. Why now: [one
line].
```

Failures the template prevents. First, the highlight reel — the monthly review that lists shipped work and skips stale work; the explicit Stale Work section is non-negotiable, and the Operator who leaves it blank should be asked to defend the claim. Second, the surprise decision — the architectural call or budget ask that lands in front of Leadership with no warning; the Decisions Ahead section is the pipeline that prevents this, and items that appear here this month often become Decisions Needed in next month’s project updates. Third, the reverse-engineered status — a monthly review written from scratch in the last week of the month, disconnected from the initiative and project updates that fed it; if the

templates above were used on cadence, this document assembles itself in under an hour.

Computed health (the RAG operating model)

Every one of the three templates carries a health field, and so far the Method has told you to *pick* it — On Track / At Risk / Off Track on a project, Green / Yellow / Red on an initiative. A pick made at 4 p.m. Friday by a tired Operator is a vibe; the perpetually-green initiative and the everything-is-fine project update are one failure in two costumes — a health signal nobody computed and nobody can defend.

Health should be computed, not picked. The Method's rule is a deterministic Red / Amber / Green ladder, evaluated top to bottom — the first matching rung sets the color:

1. **Red** — the project is Blocked (a hard blocker sits on the critical path), *or* it is past its target date with under ~25% of its issues Done.
2. **Amber** — it is past its target date with ~25–74% of issues Done, *or* any single issue has been stuck in Blocked for more than seven days.
3. **Green** — none of the above (a project that is 100% Done is Green by definition).

Three counts and a date decide the color, so two Operators looking at the same project get the same answer — a status you can recompute is one the team argues at the rule, not the case. Map the rungs onto the field Linear already gives you — Green is On Track, Amber is At Risk, Red is Off Track — and the result writes straight into the native Project Health field. This is the line to memorize: **Linear gives you the health *field*; the Method gives you the *computation rules*.** Project Health ships on every tier; the rules are yours, and without them the field is a dropdown three people fill three different ways.

Wire the rules as a small automation, not a habit. A scheduled job — or a web-hook on the relevant Linear events — runs the ladder against the API once a week, the morning the weekly project updates are due (P-19), and writes each active project's color before the Operator sits down. Between runs, two events escalate in real time: a target date passing flips the project to at least Amber, and a blocker landing on the critical path flips it to Red. The Operator never has to remember to downgrade a project; the system does, which makes this P-25 in its purest form — a convention you can check programmatically holds where one enforced by Friday willpower erodes by week four. The computed color is the

default; the Operator overrides it only on a risk the rule cannot see, and writes down why.

Computed color is necessary but not sufficient, and the gap is evidence: a project can be 100% Done and still not honestly be Green if its Done issues are empty — closed with no merged PR, no deploy log, no acceptance comment (P-6, P-7). The ladder measures whether the work *moved*; Validation measures whether it is *real*, so a mature automation checks both — issues marked Done with no evidence link leave the project not Green but unproven, which reads as Amber with a one-line reason (“3 Done issues lack release evidence”). Green that out-runs its evidence is the theater P-28 warns about; the chain holds from the color down to the merge commit in the code host (Chapter 12).

The ladder rolls up the same way. An initiative’s health is the worst color among its linked projects, escalated when those projects go quiet — an initiative riding three projects that have not posted an update in three weeks is not Green whatever their last-known colors, because the rollout has gone blind (P-21, P-28). That turns the biweekly initiative update’s Objective Health from a feeling into a function of the project layer beneath it, and keeps the monthly review’s rollout table defensible when Leadership reads it (P-27).

Where the colors are *surfaced* scales with tier; the computation does not. The per-project Project Health field is Free; trends across the portfolio — how long it sat Amber last quarter, which initiatives flip color most often — are an Insights question, and Insights is Business-tier. The org-level RAG board, where Leadership watches every initiative’s color on one screen, is a Dashboard, and Dashboards are Enterprise (Chapter 9). The rules are identical at every tier; only the reach of the result changes.

Pulse (the consumption layer)

Everything above is the *authoring* side of reporting — the Operator writing once, at altitude, on cadence. Pulse is the *delivery* side: how the people downstream receive it without opening Linear. Pulse is a Free feature, and it is what makes P-27 — Leadership consumes summaries, not raw Linear — physically true rather than aspirational.

Pulse is three things: a digest feed of project and initiative updates — every status update and health change as a readable stream instead of buried in a workspace nobody on the Leadership Team logs into; an Inbox digest, daily or weekly, that lands new updates in front of each reader on a schedule they

choose; and Pulse audio — read-aloud summaries Leadership can hear on a commute instead of read at a desk. The channel meets the reader where they are; the cadence stays exactly as authored.

The distinction that keeps Pulse from becoming noise: **Pulse is the firehose; the authored summary is the signal.** The digest carries every update as written, but it does not replace the weekly project update or the monthly operating review, because those are curated — the Operator chose what mattered, named the risk, and asked for the decision (P-21). A Leadership Team that reads only the firehose and never the authored rollup has un-hired the Operator's translation and gone back to browsing raw Linear with extra steps — the exact failure P-27 exists to prevent. Used well, Pulse keeps Leadership current between monthly reviews without their ever opening the workspace, and lets the Operator confirm an update landed without chasing a read receipt.

Friday afternoon, in practice

It is Friday at 3:40 p.m. The Operator has two active projects — call them CI/CD Backbone and Internal Tooling Integration — and three active initiatives: Platform Reliability, Operator Experience, and New Product Bring-Up. The Operator posts a project update on each project in Linear: shipped, in flight, blocked, decisions needed, next week. Each update takes eight minutes because the agent layer has pre-populated the structure from the week's Linear and code-host activity. The Operator pins a four-line summary into the team Slack channel — what shipped, one blocker, the decision now waiting on Leadership — and queues the biweekly Platform Reliability initiative update as a Monday-morning draft, because today is the second Friday of the cycle and that initiative is due. Engineer A and Engineer B each open the project updates from their phones around 5 p.m., see that the staging deploy fix shipped and that the schema migration is blocked on a Leadership decision, close the tab, and start their weekends. The Operator's Friday afternoon, before the Method: two hours. The Operator's Friday afternoon, with the Method: thirty-five minutes. The leadership note for Monday writes itself from the Friday updates because the headings match.

What reporting is for

The goal of reporting is not to perform progress; it is to surface the decisions Leadership owns. A project update that produces no decision request is fine on a green week — most weeks are green weeks and the absence of a decision is itself

information. A string of green weeks with no decision requests is a smell. The Operator is either running a project that genuinely needs nothing from Leadership for a quarter (rare), or hiding the hard parts behind tidy bullets (common). The Decisions Needed section in the project update, the risk sentence in the initiative narrative, and the Decisions Ahead section in the monthly review are the three places the system asks the Operator to be honest. The cadence — weekly, biweekly, monthly — is the part that makes the honesty habitual rather than heroic.

WHERE THIS GETS HARD

When Leadership's reporting taste has its own format — quarterly pillar dashboards, BSC-style scorecards, board narratives, OKR check-ins in a slide template the Chief of Staff inherited four years ago — and that format does not map cleanly to Linear's project and initiative primitives, the Operator ends up maintaining two reporting stacks: the Method's (for operational truth) and Leadership's (for the format they want). The translation either consumes a day a month or decays into a copy-paste that loses fidelity at the seams.

Ravenhelm assesses your operating surface and installs the Method tuned to your team, your code host, and your reporting structure. nate@ravenhelm.co

The Assessment: What This Document Doesn't Cover

PRINCIPLES CITED P-6 P-7 P-8 P-9 P-13 P-15 P-22 P-24 P-26 P-27 P-28

You are the Operator. You read the previous thirteen chapters in two sittings. You recognized your team in every one of them — the Triage queue that grew a second backlog inside it, the Validation gate you’ve been wanting to install, the project updates Leadership keeps quietly rephrasing back to you in their own format. You can install most of this. You believe in it.

You also do not have the time. And the canonical 3-person team in this document is not exactly your team. Yours has four engineers and a contract designer. Or it has two engineers and a customer-success person who files half the bug reports. Or it’s three engineers but two of them split time across a parallel product that lives in a separate GitLab group. The shape of the Method fits you. The fit of the install does not, yet.

That’s the gap this chapter exists to close.

What this document gave you

The shape. Seven operating layers — initiatives, projects, issues, views, intake, delivery evidence, analytics — and the workflow canon of nine states with Validation as the load-bearing gate (P-6). The operating surface — labels, views, templates — that turns the shape into something the workspace actually does. The code host integration that proves implementation in the place it happens (P-8) and lets Linear stop pretending to know what shipped.

The doctrine. Twenty-nine numbered principles, each with a rationale and a counter-example. The principles are the part of the Method you can carry into

any future tooling change. Linear could be replaced; the principles would survive the replacement.

The install path for four tiers. Free, Basic, Business, Enterprise — each with the specific capabilities the tier adds, the install steps the Operator can run alone, and the ceiling signals that tell you you’ve outgrown your plan. The tiering is honest: most 3-person teams live happily on Free, and the Method is designed to make that possible.

The reporting cadence. Three templates the Operator hands upward — the weekly project update, the biweekly initiative health roll-up, the monthly review. The cadence is the chain that makes Leadership’s consumption of summaries (P-27) into a system rather than a habit.

What this document deliberately did not give you

Tuning to your specific team composition. The Method assumed a 3-person team: the Operator, Engineer A, Engineer B, and Leadership Team upstream. Your team is 5, or 12, or a 3-person team that contracts with a fourth specialist two days a week. The view set is roughly the same. The labels that fit your team are not. *Waiting on Operator* changes meaning when there are two Operators on rotation. *Agent Delegation Queue* changes shape when the contractor is a human, not an agent, but is delegated work through the same routing pattern. These are iterations, not copy-pastes.

Tuning to your specific code host stack. The Method assumed GitHub primary with a GitLab parallel. Your team uses both, plus a private Gitea mirror for the on-prem build, plus a Bitbucket repo for the legacy product you can’t decommision yet. The Validation → Done transition needs evidence from each one (P-9). Webhook design, branch naming conventions, and release-pipeline configuration are bespoke to the stack you actually have.

Tuning to your leadership team’s reporting taste. The three templates in Chapter 13 are a starting point. The format Leadership actually consumes — pillar dashboards, OKR scorecards, board-pack narratives, a Slack-channel weekly with three bullets — is a translation layer on top of the Method’s evidence chain (P-28). Project updates feed initiative updates which feed the monthly review; the format of the monthly review depends on what your leadership reads without resentment. That is a design decision, and it is yours.

Tuning to your intake sources. The Method named Triage, Customer Requests, and Asks. Your team’s intake includes a support ticket queue from a separate

help desk, a Sentry stream that fires three times a day, a Slack alerts channel, customer-success escalations forwarded by email, and an open-source GitHub Issues queue with weekly contributions. Each one needs a designed entry point that lands somewhere the Method can pick it up (P-22). None of those entry points existed in the canonical example.

Designing the agent layer (if you have one or want one). The Augmentation Surface boxes throughout this document named where an agent layer extends the Method — triage classification, project update drafting, evidence reconciliation, stale-work detection. What an agent layer *does* is described. *How* to build it — what it reads, what it writes, when it routes a decision into the Waiting on Operator view, what acceptance criteria its outputs need before they're trusted (P-15) — is Ravenhelm's product, not configuration you can crib from a document.

If you are the Operator on a 3-person team with a single primary repo and a Leadership Team that already reads the three templates without translation, this document is usable as-is and your install is mostly mechanical. If your team has scaled past three engineers, or your leadership team has a reporting format the templates don't match, or your code host stack includes more than GitHub and one parallel — the install you do alone will be roughly seventy percent of the Method. The remaining thirty percent is the tuning the assessment delivers.

The Ravenhelm Operating Assessment

The engagement is shaped as five sequential steps. Each step has a defined output. The Operator is the counterparty throughout — the assessment does not negotiate with Leadership for you and does not bypass the role of the person who owns priority, taste, risk, and real-world commitments (P-13).

1. Discovery interview. Sixty to ninety minutes with the Operator and one Leadership stakeholder. The goal is to hear the current state in your language: where the workspace is today, where the friction is, where the next ninety days are aimed, what the Leadership Team is asking for that the workspace is not delivering. No diagnosis yet. The output is a short written brief — what the engagement believes is true about the team, returned for correction before the audit starts.

2. Workspace audit. Ravenhelm runs the gap-analysis methodology against the current Linear workspace. The audit is read-only: nothing in the workspace changes during this step. The output is a written audit naming what already aligns with the Method, what conflicts with it, and what is missing. The methodology is the same one the Method itself was built from — adapted from the gap-

analysis pattern in the Ravenhelm canon and applied to your specific workspace inventory.

3. Gap classification against the Method. Each gap from the audit is categorized into one of four buckets: *install gap* (the Method is silent and a decision is needed for your team), *drift gap* (a Method principle is being violated and the workspace needs intervention to come back into alignment), *tier gap* (the team has outgrown its current Linear plan and the workspace is being constrained by ceiling signals), or *tooling gap* (the team needs an integration the Method does not currently prescribe — a help desk, a Sentry stream, a second code host). The classification matters because each bucket has a different fix path and a different cost.

4. Phased install plan. Adapted from the Method's own seven-phase install pattern. Each phase names what changes, what evidence proves the phase landed (P-7), who owns each install task, and how long the phase takes. Phases are sized for your team's actual capacity, not theoretical capacity — the install respects the seventy-percent rule (P-26) the same way real delivery does. The plan also names what the engagement recommends *not* installing. The overbuild test (P-24) applies as hard to the Method as it does to any other tooling choice: if a Method element fails four-of-five for your team, the install plan leaves it out.

5. Ongoing maturity coaching. Adapted from the long-horizon maturity pattern in the Method's canon. For the first quarter after install, the engagement provides monthly Operator coaching: stuck-decision review, drift detection in the workspace, the next-tier readiness call when ceiling signals start appearing. Coaching is optional for the engagement — named here so the reader knows it exists and can choose. The Method is built to keep working without external help; the coaching is for the team that wants someone else carrying the operating-model maintenance load while the Operator focuses on the work itself.

The five steps run in order. The discovery and audit can be completed inside two weeks; the install plan lands inside four weeks of engagement start; the install itself runs against your team's capacity, typically four to eight weeks for a Business-tier install at a 5-to-12-person team. The coaching, if chosen, begins the month after install close.

Engaging

The Ravenhelm Operating Assessment is the engagement. The contact path is `nate@ravenhelm.co`.

The Operations-Team Variant

OPT-IN VARIANT	PRINCIPLES CITED	P-1	P-2	P-3	P-10	P-11	P-13	P-16	P-17
P-19	P-20	P-21	P-22	P-24	P-25	P-27	P-28	P-29	

It is Monday, and the Operator runs a 3-person internal-services team — process automation and system integrations for the rest of the company. The board on the screen did not come from a roadmap the team wrote. It came from other people. Engineer A spent Friday on an integration a director asked for in a hallway, verbally, with no ticket. Engineer B is three weeks into an automation nobody formally requested, because a manager asked in a Slack DM and saying no felt rude. Two separate VPs each believe their request is next in line; neither is written down anywhere the other can see. There is no front door, so every door is the front door, and the Operator spends the first hour of every week reconstructing what the team even agreed to do.

That is the problem this appendix exists to solve, and it is a different problem than the rest of the Method addresses.

Supply-side and demand-side — and who should skip this appendix

The Method’s default reader is a **supply-side** team: a product-dev team that builds what is on its own board. Work originates inside the team, from a roadmap the Operator and Leadership set. Intake is a thin layer — the occasional bug report or feature idea the team chooses to accept — and the Triage-and-Asks machinery in Chapter 8 covers it completely. Everything that matters happens *after* an item is on the board.

A Center of Excellence, an operations team, a PMO, an internal platform or shared-services group is **demand-side**. The organization requests work *through* it. Its board is populated almost entirely by other people’s asks, and the hardest,

most failure-prone part of its operating model is not delivery — it is the seam where a request becomes committed work. For a demand-side team, intake *is* the product.

Here is the load-bearing claim of this appendix: the Method's **back half is roughly 80% identical for both kinds of team and is not restated here**. Once a request is accepted, it becomes an issue in a project under an initiative (**P-2**, **P-3**); it runs the nine-state workflow canon; it hits the Validation gate and must produce evidence before Done; it rolls up through the weekly-biweekly-monthly reporting cadence in Chapter 13; its health is computed, not picked. None of that changes. What changes is the **front half** — how work *enters* and is *governed* — and that is all this appendix installs. It swaps the demand-side front half in for teams that need it and leaves everything downstream exactly as the core chapters describe.

State the negative plainly, because it is the more common case: **a pure product-dev team does not need this appendix**. If the majority of your board originates from your own roadmap rather than from other people's requests, the intake pipeline, the service catalog, the service SLAs, and the field rigor below are overhead that fails the overbuild test (**P-24**). Install this variant only when you are answering demand you did not generate.

Intake as a first-class layer

A demand-side team cannot treat intake as a side effect of Triage. It needs a formal flow that every request travels, visibly, on the record:

1. **Request** — captured through the front door (below), never verbally, never in a DM. A request that exists only in a hallway conversation does not exist (**P-1**).
2. **Intake Review** — the request has been seen and is being sized and clarified, but has not been accepted. Nothing is committed here.
3. **Groomed** — the request is understood well enough to accept or decline: scope is clear, the objective it serves is named, the requester has confirmed what they actually want.
4. **Accepted** — the request converts to a project or issue, gets a home in an initiative, and is assigned a target release version (**P-29**). Only here does it enter the nine-state delivery canon at Backlog.

The stage worth building deliberately is **Intake Review**, and its purpose is a metric. A product-dev team would never add it — the workflow canon is nine states and a tenth is a smell (**P-10**), and a supply-side team should reach for a `stage:*`

label before a new state every time (P-11). A service team earns the exception, because the number Intake Review produces is the number a service team is judged on: **dwelt time** — how long a request sits between arriving and being accepted. That is the seven-day Triage clock (P-20) generalized into a service-health metric. Intake Review lives in the intake pipeline *before* Backlog, so it never touches the nine delivery states; what it adds is a timestamp pair the Operator can report. When average intake dwelt climbs from two days to nine, the team has a capacity problem its requestors are already feeling — and now it can see it before the complaints arrive, instead of after (P-20).

The Service Catalog

The cure for “every door is the front door” is one published door. The team defines a small set of named **request types** and publishes them as its service catalog — the front-of-house menu of what the team does and how to ask for it. Generic examples a CoE tends to carry: a **process-automation candidate**, an **enhancement request** against a system the team owns, an **integration build**, and a **governance exception**. The catalog is the difference between a team that receives requests and a team that receives *legible* requests.

Each request type maps to a Linear **Asks** template (Chapter 8) built from the operating surface’s own label, view, and template mechanics (Chapter 5) — it pre-tags the team, the target project, and the labels the type always carries, and presents an intake form whose required fields are fixed for that type. Submitting the form auto-creates a triaged issue that already has its structure — the Operator opens a sized, labeled, routable request instead of “can you help with something?” from a channel they do not watch. This is P-17 doing its job at the front door: templates lower the cost of doing the right thing, so the right thing is the path of least resistance. It replaces the ad-hoc hallway-Slack-email intake that was drowning the Operator on Monday, and it does so without the Operator having to police anyone — the form is easier than the DM (P-22: a request is captured and triaged, never converted straight to scope by whoever asked loudest).

AUGMENTATION SURFACE • Intake Classification

Where an agent layer makes this work better: reading each inbound request as it lands, matching it to a catalog request type, pre-filling the intake form, flagging the missing required fields, drafting a first-response acknowledgment inside the service SLA, and tracking intake dwell so the queue self-reports (P-20). A demand-side team runs two agent roles, not one — an *intake* agent that classifies and routes what arrives, distinct from any *build* agent that executes what was accepted — and keeping them separate keeps classification from quietly becoming commitment.

What the PM keeps: the accept-or-decline call and the charter decision. Customer and requester feedback is signal, not scope (P-22); priority, taste, risk, and real-world commitments do not delegate (P-13). The agent sizes and routes the request; the Operator decides whether the team takes it.

Governance cadence

The core reporting cadence in Chapter 13 — weekly project updates (P-19), bi-weekly initiative updates, a monthly operating review (P-21) — holds unchanged. A demand-side team extends it at both ends rather than replacing it, and the discipline is to frame these as extensions, not a parallel bureaucracy that fails the overbuild test (P-24).

At the fast end, a **daily** rhythm: a short standing pass over Intake Review, so no request sits un-acknowledged past its first-response clock, and the day's accepted work is visible. At the slow end, a **Quarterly Business Review** — the QBR — where the team reports to its sponsors on what it accepted, what it delivered, intake dwell and SLA-breach trends, and what it is declining and why. The QBR is the monthly review (P-21) compressed one more level, aimed at the people who fund the team rather than the people who read weekly updates.

Two governance gates matter for a team that spends other people's mandate:

- **A charter-approval gate.** Before a project leaves intake and starts, a named sponsor signs off on it — the objective it serves, the rough cost, the trade-off against what the sponsor's request displaces. This is P-13 rendered as a gate: a real-world commitment does not get made because a form was submitted; it gets made because the Operator (Chapter 4) and an accountable sponsor both agreed to it. The gate is what stops the loudest requester from setting the team's priorities by volume (P-22).
- **RACI on stakeholder-heavy initiatives.** When an initiative touches several departments, name who is Responsible, Accountable, Consulted, and Informed

on it, in the initiative document. A supply-side team rarely needs this; a demand-side team with five stakeholders per initiative needs it to keep “who approves this” from being re-litigated every update.

Service SLAs

This is the commitment a service team owes that a product-dev team owes no one. A supply-side team answers to its own roadmap; a demand-side team answers to requestors who are waiting, and it owes them clocks. Two, specifically, layered onto the Business-tier SLAs in Chapter 8:

- **Time-to-first-response** — how long from a request landing to the requester hearing back, even if the answer is “received, in Intake Review, decision by Thursday.” This is the clock that most cheaply buys trust, because most requester frustration is silence, not delay.
- **Time-to-resolution** — how long from acceptance to Done, tracked per request type, because an integration build and a governance exception do not carry the same promise.

Both attach to Triage and to Asks-sourced items using Linear SLAs, which are a Business-tier capability (Chapter 8). Alongside them, keep the escalation rule the core already implies and make it explicit for a service team: **an item blocked more than N days escalates** to the Operator and, past a second threshold, to the sponsor. As Chapter 8 warns, calibrate these against real history — an SLA on every request produces a queue where everything is breaching and nothing is urgent, and an untrusted SLA is worse than none (P-25). The clock exists to make silence impossible, not to manufacture false alarms.

CoE-grade field discipline

A demand-driven team needs rigor at issue creation that a 3-person product team can skip, and the reason is the demand itself: when work arrives from dozens of requesters across the org, the only way a report rolls up honestly is if every item was captured to the same shape. A supply-side team of three shares context in its heads; a service team’s context lives in the fields or it does not live at all.

Three rules, stricter than core:

- **The Rule of One.** Each parent issue carries exactly one required objective or OKR label — one, not zero and not three — plus a matched set of required

single-select fields for its type. One objective per parent is what makes the rol-up unambiguous: every unit of work answers to exactly one stated goal, so the QBR can total effort by objective without double-counting or orphans (P-2, P-16).

- **A per-issue-type required-field matrix**, enforced as a hard create-time block, not a habit. Because the rule is checked programmatically at creation rather than policed after the fact, it actually holds (P-25):

REQUEST TYPE	REQUIRED FIELDS AT CREATION
Process-automation candidate	requester, sponsor, objective label, current-effort estimate, expected saving
Enhancement request	requester, target system, objective label, priority, acceptance criteria
Integration build	requester, source system, target system, objective label, data-sensitivity
Governance exception	requester, policy referenced, objective label, approver, expiry date

- **Mandatory objective and initiative linkage** per parent — no accepted parent issue exists without a project and an initiative above it (P-3). A supply-side team can tolerate a loose issue for a day; a demand-side team cannot, because a request with no objective above it is a favor, and a board full of favors is how a service team loses the ability to say what it is for.

The discipline is deliberately heavier than the core, and the overbuild test (P-24) is the check on it: each required field must earn its place by feeding a decision, a report, or the charter gate. A field nobody reads is a tax on every requester, and a service team’s requesters abandon a form faster than a teammate does.

Two optional label axes

Two label axes serve views (Chapter 5) a demand-side team tends to want, and both are strictly optional — add either only when a real view needs it, never speculatively (P-16, P-24).

- **layer:** — capability / delivery / impact. The plain-language version is *building the factory / running the factory / measuring the value*: capability is the team’s own tooling and platform build-out, delivery is the run-the-factory work of fulfilling requests, impact is the measurement and value-tracking work. The view this serves is the mix question every service team’s sponsor

eventually asks — how much of the quarter went to building versus running versus proving worth — which is unanswerable without the axis and trivial with it.

- **customer:** — internal / external . This splits requests by who they ultimately serve, so the team can show internal-facing versus external-facing load on one filter. It matters most when a CoE straddles both — automating an internal finance process one week and shipping something a paying customer touches the next, at very different evidence bars.

Neither axis is prescribed. Each is justified the moment its view is one Leadership or a sponsor asks for repeatedly, and deleted the moment no view cites it (P-16).

Computed RAG governance

A CoE reports health to its executives on a rolled-up RAG dashboard, and the operating model for that health is already written — this appendix does not restate it. The deterministic Red / Amber / Green ladder, the rule that health is computed rather than picked, the escalation on stale rollups, and the split between the health *field* Linear gives you and the computation *rules* the Method gives you all live in Chapter 13, and they apply here without modification (P-28). The demand-side additions feed the same computation rather than replacing it: intake dwell time (from Intake Review) and SLA-breach counts become inputs the executive board watches next to project and initiative color.

Where that board *lives* is the only tier-dependent part. A per-project health color is Free; portfolio trends are an Insights question and Insights is Business; the org-level RAG board — every initiative's color on one screen for the sponsors — is a Dashboard, and Dashboards are an Enterprise capability (Chapter 9). Leadership reads that board, not the workspace beneath it (P-27); the Operator's job is to keep the layers under it honest so the color the sponsors see is one the team can defend all the way down to the evidence (P-28).

WHERE THIS GETS HARD

The organization that wants a real front door — a published catalog, enforced intake, service SLAs the requesters trust, a charter gate that survives a VP walking up to an engineer's desk — is asking a 3-person team to hold a governance boundary against people who outrank it, using rigor (the Rule of One, the required-field block, the RACI) that is only worth the friction if it is installed and calibrated correctly the first time. Install the field discipline too loose and the reports lie; too tight and the requesters route around the form and you are back to the hallway. The catalog ships in an afternoon; the boundary that makes it stick takes a season and a mandate the team usually does not have alone.

Ravenhelm assesses your operating surface and installs the Method tuned to your team, your code host, and your reporting structure. nate@ravenhelm.co

BACK MATTER

RLOM Community License

Version 1.0

Copyright © 2026 Ravenhelm LLC. All rights reserved except as expressly granted below.

PERMITTED USES

Permission is hereby granted, without charge, to any person or organization obtaining a copy of the Ravenhelm Linear Operating Method (“RLOM”):

1. **Informational & Educational Use:** To read, download, store, copy, and redistribute verbatim, unmodified copies of RLOM for non-commercial, personal, or educational purposes.
2. **Internal Implementation:** To adopt, use, and implement the RLOM Method and its principles within your own organization for internal business operations and engineering workflow management.
3. **Internal Adaptations:** To create internal documentation, workspace configurations, issue templates, and operational runbooks necessary to execute and adapt RLOM within your own organization.

RESTRICTIONS & RESERVED RIGHTS

Except as expressly authorized above, you may not, without prior written permission from Ravenhelm LLC:

1. **Commercial Distribution:** Sell, rent, sublicense, monetize, or commercially distribute RLOM or substantial portions thereof in any medium.
2. **Derivative Works:** Publish, distribute, or make publicly available modified versions, adaptations, translations, or derivative works based on RLOM.
3. **Third-Party Services:** Offer, market, or provide commercial consulting, implementation, assessment, audit, training, coaching, or certification services to third parties using or referencing the RLOM name, methodology, or materials.
4. **Commercial Packaging:** Incorporate RLOM or substantial portions of it into a commercial product, SaaS offering, methodology framework, publication, or fee-based training curriculum.
5. **Brand & Endorsement:** Use the names, trademarks, or service marks of “Ravenhelm” or “RLOM” in any manner that suggests sponsorship, affiliation, partnership, certification, or endorsement by Ravenhelm LLC.

DISCLAIMER OF WARRANTIES

RLOM IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES, OR OTHER LIABILITY ARISING FROM, OUT OF, OR IN CONNECTION WITH THE METHOD OR THE USE OR OTHER DEALINGS IN IT.

COMMERCIAL LICENSING, IMPLEMENTATION & INQUIRIES

For commercial implementation partnerships, enterprise assessments, third-party consulting rights, or custom licensing inquiries, contact:

Nathan Walker

Ravenhelm LLC

Email: nate@ravenhelm.co

Web: <https://ravenhelm.co>



Ravenhelm Linear Operating Method – v1.0.2

© 2026 Ravenhelm LLC. All rights reserved.

Set in Source Serif 4 and IBM Plex Mono.